Covid 19 Google Repository Data Analysis Report

2025-09-19

Part 1: Introduction to the Data Set

The data set is taken from Google's data repository on the Covid-19 pandemic. More specifically, 4 different datasets were selected; epidemiology, demographics, health and economics data. Before introducing the specifics of each data set, the code below goes through the process of importing, joining and cleaning the three datasets. This is done using the readr and tidyverse package. All of the data sets are loaded into their own data frame as shown below.

```
pacman::p_load(magrittr, tidyverse, ggplot2, dplyr, knitr)

demographics <- read.csv("data/demographics.csv")
economy <- read.csv("data/economy.csv")
epidemiology <- read.csv("data/epidemiology.csv")
health <- read.csv("data/health.csv")
index <- read.csv("data/index.csv")

head(epidemiology, 3)
```

```
##         date location_key new_confirmed new_deceased new_recovered new_tested
## 1 2020-01-01           AD             0            0            NA         NA
## 2 2020-01-02           AD             0            0            NA         NA
## 3 2020-01-03           AD             0            0            NA         NA
##   cumulative_confirmed cumulative_deceased cumulative_recovered
## 1                    0                   0                  NA
## 2                    0                   0                  NA
## 3                    0                   0                  NA
##   cumulative_tested
## 1                NA
## 2                NA
## 3                NA
```

```
head(demographics, 3)
```

```
##   location_key population population_male population_female population_rural
## 1           AD      77265           58625             55581             9269
## 2           AE    9890400         6836349           3054051          1290785
## 3           AF   38928341        19976265          18952076         28244481
##   population_urban population_largest_city population_clustered
## 1            67873                     NA                   NA
## 2          8479744                2833079              5914068
## 3          9797273                4114030              4114030
##   population_density human_development_index population_age_00_09
```

```
## 1            164.394                   0.858                9370
## 2            118.306                   0.863             1011713
## 3             59.627                   0.498            11088732
##   population_age_10_19 population_age_20_29 population_age_30_39
## 1                12022                10727                12394
## 2               842991              2149343              3169314
## 3              9821559              7035871              4534646
##   population_age_40_49 population_age_50_59 population_age_60_69
## 1                21001                20720                14433
## 2              1608109               797913               242707
## 3              2963459              1840198              1057496
##   population_age_70_79 population_age_80_and_older
## 1                 8657                        4881
## 2                55884                       12426
## 3               480455                      105925
```

```
head(economy, 3)
```

```
##   location_key      gdp_usd gdp_per_capita_usd human_capital_index
## 1           AD   3154057987              40886                  NA
## 2           AE 421142267937              43103               0.659
## 3           AF  19101353832                502               0.389
```

```
head(health, 3)
```

```
##   location_key life_expectancy smoking_prevalence diabetes_prevalence
## 1           AD              NA                33.5                 7.7
## 2           AE          77.814                28.9                16.3
## 3           AF          64.486                  NA                 9.2
##   infant_mortality_rate adult_male_mortality_rate adult_female_mortality_rate
## 1                   2.7                        NA                          NA
## 2                   6.5                    69.555                      44.863
## 3                  47.9                   237.554                     192.532
##   pollution_mortality_rate comorbidity_mortality_rate hospital_beds_per_1000
## 1                       NA                         NA                     NA
## 2                     54.7                       16.8                     NA
## 3                    211.1                       29.8                    0.5
##   nurses_per_1000 physicians_per_1000 health_expenditure_usd
## 1          4.0128              3.3333             4040.78662
## 2          5.7271              2.5278             1357.01746
## 3          0.1755              0.2782               67.12265
##   out_of_pocket_health_expenditure_usd
## 1                           1688.12146
## 2                            256.03449
## 3                             50.66591
```

As a brief preview the epidemiology data set captures Covid cases, deaths and recovery statistics for every day from 2020 to 2022 with the subject being different regions labelled by the location key. The demographics' data set captures population statistics, the economics data set captures economical statistics such as GDP and human capital index, and finally the health dataset records healthcare and quality of living statistics of different regions listed within the data. These data sets captures the regions' average statistic from 2020-2022. The index data set is imported as a guide to rename the different locations with their respective location keys later on.

Combining all datasets and selecting useful variables

Here all of the data frames of th datasets are joined together onto a single data frame. The main data frame is epidemiology and the remaining data frames are joined onto it via the location_key variable; the final data frame with all of the data sets is called uncleaned_df. Afterwards the data frame is transformed into a tibble for cleaner visualisation and better functionality with the R packages that will be used later on such as ggplot, dplyr and tidyr.

```
uncleaned_df <- left_join(epidemiology, demographics, by = "location_key")

uncleaned_df <- left_join(uncleaned_df, economy, by="location_key")

uncleaned_df <- left_join(uncleaned_df, index, by="location_key")

uncleaned_df <- left_join(uncleaned_df, health, by="location_key")

uncleaned_df <- as.tibble(uncleaned_df)
```

```
## Warning: `as.tibble()` was deprecated in tibble 2.0.0.
## i Please use `as_tibble()` instead.
## i The signature and semantics have changed, see `?as_tibble`.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
uncleaned_df
```

```
## # A tibble: 12,525,825 x 58
##    date       location_key new_confirmed new_deceased new_recovered new_tested
##    <chr>      <chr>                <int>        <int>         <int>      <int>
##  1 2020-01-01 AD                       0            0            NA         NA
##  2 2020-01-02 AD                       0            0            NA         NA
##  3 2020-01-03 AD                       0            0            NA         NA
##  4 2020-01-04 AD                       0            0            NA         NA
##  5 2020-01-05 AD                       0            0            NA         NA
##  6 2020-01-06 AD                       0            0            NA         NA
##  7 2020-01-07 AD                       0            0            NA         NA
##  8 2020-01-08 AD                       0            0            NA         NA
##  9 2020-01-09 AD                       0            0            NA         NA
## 10 2020-01-10 AD                       0            0            NA         NA
## # i 12,525,815 more rows
## # i 52 more variables: cumulative_confirmed <int>, cumulative_deceased <int>,
## #   cumulative_recovered <int>, cumulative_tested <dbl>, population <int>,
## #   population_male <int>, population_female <int>, population_rural <int>,
## #   population_urban <int>, population_largest_city <int>,
## #   population_clustered <int>, population_density <dbl>,
## #   human_development_index <dbl>, population_age_00_09 <int>, ...
```

Note that one limitation with the data is that although the epidemiology data set is a time series dataset, the economy and demographics data are both cross sectional data sets and so when joining the data sets the population and economic variable's values per each date observation should be taken as an average or a bench mark and not the accurate value for that date.

The tibble above shows this data frame currently contains 12,525,825 observations and 58 variables. That is too much data for the purpose of this investigation, so this data frame will be simplified as follows. The epidemiology data set not only contains statistics for just countries but also the country and its individual regions are represented by separate rows within the data. Country level observations are represented with a two letter location key e.g., AF for Afghanistan and a region in Afghanistan would be represented by the two letters of the country with another three letter code e.g., AF_BAL. To simplify the data the investigation will only be looking at country level observations; hence to code below specifically filters for location keys with two letters. Otherwise the code below also selects specific variables of interest for further investigation.

```
uncleaned_df1 <- uncleaned_df %>% filter(nchar(location_key) == 2) %>%
  select(date,
         country_name, cumulative_confirmed, cumulative_deceased, gdp_usd,
         gdp_per_capita_usd,
         population,
         population_rural,
         population_urban,
         population_density,
         population_age_60_69,
         population_age_70_79,
         population_age_80_and_older,
         health_expenditure_usd,
         smoking_prevalence,
         life_expectancy,
         nurses_per_1000)

#Changing the date column to a date type variable
uncleaned_df1$date <- as.Date(uncleaned_df1$date, format="%Y-%m-%d")

summary(uncleaned_df1)
```

```
##       date            country_name       cumulative_confirmed
##  Min.   :2019-12-31   Length:226892      Min.   :       0
##  1st Qu.:2020-09-03   Class :character   1st Qu.:     616
##  Median :2021-05-08   Mode  :character   Median :   18301
##  Mean   :2021-05-07                      Mean   :  867843
##  3rd Qu.:2022-01-09                      3rd Qu.:  249420
##  Max.   :2022-09-16                      Max.   :92440495
##                                          NA's   :94
##  cumulative_deceased    gdp_usd          gdp_per_capita_usd   population
##  Min.   :      0     Min.   :4.727e+07   Min.   :   261     Min.   :5.000e+01
##  1st Qu.:      8     1st Qu.:5.920e+09   1st Qu.:  2229     1st Qu.:4.375e+05
##  Median :    243     Median :2.759e+10   Median :  6966     Median :5.518e+06
##  Mean   :  13830     Mean   :4.312e+11   Mean   : 17732     Mean   :3.376e+07
##  3rd Qu.:   3871     3rd Qu.:2.180e+11   3rd Qu.: 23139     3rd Qu.:2.141e+07
##  Max.   :1005195     Max.   :2.137e+13   Max.   :185829     Max.   :1.439e+09
##  NA's   :267         NA's   :28611       NA's   :28611
##  population_rural    population_urban    population_density
##  Min.   :        0   Min.   :     5464   Min.   :    0.138
##  1st Qu.:   205163   1st Qu.:   417765   1st Qu.:   38.420
##  Median :  1808321   Median :  3907243   Median :   93.100
##  Mean   : 16078813   Mean   : 20170912   Mean   :  393.172
##  3rd Qu.:  9989403   3rd Qu.: 11116711   3rd Qu.:  234.700
##  Max.   :895386226   Max.   :842933962   Max.   :26338.255
##  NA's   :18741       NA's   :18741       NA's   :3948
```

```
##  population_age_60_69 population_age_70_79 population_age_80_and_older
##  Min.   :       76    Min.   :      28     Min.   :      11
##  1st Qu.:    33573    1st Qu.:   14721     1st Qu.:    5428
##  Median :   372086    Median :  182626     Median :   58463
##  Mean   :  2582082    Mean   : 1360035     Mean   :  632374
##  3rd Qu.:  1278758    3rd Qu.:  680922     3rd Qu.:  319044
##  Max.   :151663905    Max.   :71494305     Max.   :26618103
##  NA's   :2961         NA's   :2961         NA's   :2961
##  health_expenditure_usd smoking_prevalence life_expectancy nurses_per_1000
##  Min.   :   19.43       Min.   : 2.00      Min.   :52.80   Min.   : 0.074
##  1st Qu.:   82.08       1st Qu.:14.00      1st Qu.:67.67   1st Qu.: 1.298
##  Median :  332.57       Median :21.80      Median :74.13   Median : 3.074
##  Mean   : 1084.76       Mean   :21.64      Mean   :72.80   Mean   : 4.408
##  3rd Qu.: 1112.30       3rd Qu.:28.00      3rd Qu.:78.29   3rd Qu.: 6.428
##  Max.   :10246.14       Max.   :47.00      Max.   :85.42   Max.   :19.461
##  NA's   :44262          NA's   :83743      NA's   :26637   NA's   :50184
```

Dealing with NAs

All of the ranges and values looks fine for all of the variables, however there are a some missing values present. The code below shows the selected variables and their proportion of missing data

```
colMeans(is.na(uncleaned_df1)) * 100   #percentage missing per column
```

```
##                         date                country_name
##                    0.0000000                   0.0000000
##         cumulative_confirmed          cumulative_deceased
##                    0.0414294                   0.1176771
##                      gdp_usd           gdp_per_capita_usd
##                   12.6099642                  12.6099642
##                   population             population_rural
##                    0.0000000                   8.2598769
##             population_urban           population_density
##                    8.2598769                   1.7400349
##         population_age_60_69         population_age_70_79
##                    1.3050262                   1.3050262
## population_age_80_and_older      health_expenditure_usd
##                    1.3050262                  19.5079597
##           smoking_prevalence             life_expectancy
##                   36.9087495                  11.7399468
##              nurses_per_1000
##                   22.1180121
```

For this investigation, it is determined that if missing data makes up of less than 10% of the variable's observations then it can be seen as random noise. Therefore, variables that have a higher proportion of missing values are considered to be affected by systematic missingness. For each of these variables, the observations are grouped within their continental region using the R countrycode package. The regional average is then used as the imputed value for the missing data. The main limitation with this approach is that many values could easily be understated or overstated, for example, countries that had missing GDP most likely are poorer countries that does not report GDP, therefore using the average of that countries' region to represent its missing data is not a true representation and therefore is something to be considered when interpreting future results.

```r
#Imputing NAs
pacman::p_load(countrycode)

uncleaned_df1$region <- countrycode(uncleaned_df1$country_name,
                          origin = "country.name",
                          destination = "region")
```

```
## Warning: Some values were not matched unambiguously: Mayotte, Micronesia, Réunion, Saint Helena, Wall
```

```r
#Manually filling region since these countries
#are not listed in countrycode package
uncleaned_df1 <- uncleaned_df1 %>% mutate(region = case_when(
  country_name == "Mayotte" ~ "Sub-Saharan Africa",
  country_name == "Kosovo" ~ "Europe & Central Asia",
  country_name == "Micronesia" ~ "East Asia & Pacific",
  country_name == "Réunion" ~ "Sub-Saharan Africa",
  country_name == "Saint Helena" ~ "Sub-Saharan Africa",
  country_name == "Wallis and Futuna" ~ "East Asia & Pacific",
  country_name == "Western Sahara" ~ "Middle East & North Africa",
  TRUE ~ region
))

uncleaned_df1 %>% distinct(region)
```

```
## # A tibble: 7 x 1
##   region
##   <chr>
## 1 Europe & Central Asia
## 2 Middle East & North Africa
## 3 South Asia
## 4 Latin America & Caribbean
## 5 Sub-Saharan Africa
## 6 East Asia & Pacific
## 7 North America
```

```r
uncleaned_df2 <- uncleaned_df1 %>%
  group_by(region) %>%
  mutate(
    across(
      c(health_expenditure_usd, smoking_prevalence, life_expectancy, gdp_per_capita_usd, gdp_usd, nurses
      ~ ifelse(is.na(.x), mean(.x, na.rm = TRUE), .x)
    )
  ) %>%
  ungroup()

#Finding number of rows with at least 1 missing/NA value.

colSums(is.na(uncleaned_df2))
```

```
##                         date                 country_name
##                            0                            0
##         cumulative_confirmed          cumulative_deceased
```

```
##                          94                         267
##                    gdp_usd          gdp_per_capita_usd
##                          0                           0
##                 population            population_rural
##                          0                       18741
##            population_urban          population_density
##                       18741                        3948
##        population_age_60_69        population_age_70_79
##                       2961                        2961
## population_age_80_and_older      health_expenditure_usd
##                       2961                           0
##          smoking_prevalence             life_expectancy
##                          0                           0
##             nurses_per_1000                      region
##                          0                           0
```

The remaining missing values are seen as random noise and therefore is omitted. Assigning the final data frame as covidDf.

```
covidDf <- na.omit(uncleaned_df2)
covidDf$region <- as.factor(covidDf$region)
covidDf
```

```
## # A tibble: 207,149 x 18
##    date       country_name cumulative_confirmed cumulative_deceased    gdp_usd
##    <date>     <chr>                       <int>               <int>      <dbl>
##  1 2020-01-01 Andorra                         0                   0 3154057987
##  2 2020-01-02 Andorra                         0                   0 3154057987
##  3 2020-01-03 Andorra                         0                   0 3154057987
##  4 2020-01-04 Andorra                         0                   0 3154057987
##  5 2020-01-05 Andorra                         0                   0 3154057987
##  6 2020-01-06 Andorra                         0                   0 3154057987
##  7 2020-01-07 Andorra                         0                   0 3154057987
##  8 2020-01-08 Andorra                         0                   0 3154057987
##  9 2020-01-09 Andorra                         0                   0 3154057987
## 10 2020-01-10 Andorra                         0                   0 3154057987
## # i 207,139 more rows
## # i 13 more variables: gdp_per_capita_usd <dbl>, population <int>,
## #   population_rural <int>, population_urban <int>, population_density <dbl>,
## #   population_age_60_69 <int>, population_age_70_79 <int>,
## #   population_age_80_and_older <int>, health_expenditure_usd <dbl>,
## #   smoking_prevalence <dbl>, life_expectancy <dbl>, nurses_per_1000 <dbl>,
## #   region <fct>
```

Introducing the final data set

For this investigation the covidDf data set on Covid-19 will be used. The subject of the data set are countries and that data set itself captures different basic statistics regarding the affect of Covid, and the countries' population, GDP, number of nurses per 1000 people etc... from 2020 to 2022. There are 207,149 rows and 18 columns with the majority of the variables being numerical, the countries' region is the only factor and country name is the only character variable.

The main goal of this investigation is to explore the relationship between the number of Covid cases and Covid deaths of a country and different factors like population demographics, economy and healthcare.

Therefore, these variables selected were because they could represent these factors, propose an interesting relationship with Covid numbers as well as not having too many missing values. For further detail of each of the variables in the data set refer to the table below.

```
main_stats <- c(
  "date", "country_name", "cumulative_confirmed", "cumulative_deceased", "gdp_usd",
  "gdp_per_capita_usd", "population", "population_rural", "population_urban",
  "population_density", "population_age_60_69", "population_age_70_79",
  "population_age_80_and_older",  "health_expenditure_usd", "smoking_prevalence",
  "life_expectancy", "nurses_per_1000", "region"
)

des <- c("date (YYYY-MM-DD)", "Name of country (211 different countries)",
         "Cumulative sum of cases confirmed after positive test",
         "Cumulative sum of deaths from a positive COVID-19 case",
         "Growth Domestic Product in USD", "GDP per Capita in USD",
         "Total count of people", "Population in rural area",
         "Population in urban area",
         "Population per squared kilometer of land area",
         "Estimated population aged bewteen 60 to 69",
         "Estimated population aged bewteen 70 to 79",
         "Estimated population aged bewteen 80 and older",
         "Government health expenditure per capita", "% of smokers in population",
         "Average years an individual is expected to live",
         "Nurses per 1000 people", "Regional classification of the countries")

stats_table <- data.frame(Main_Stats = main_stats, Descriptions = des)
knitr::kable(stats_table)
```

Main_Stats	Descriptions
date	date (YYYY-MM-DD)
country_name	Name of country (211 different countries)
cumulative_confirmed	Cumulative sum of cases confirmed after positive test
cumulative_deceased	Cumulative sum of deaths from a positive COVID-19 case
gdp_usd	Growth Domestic Product in USD
gdp_per_capita_usd	GDP per Capita in USD
population	Total count of people
population_rural	Population in rural area
population_urban	Population in urban area
population_density	Population per squared kilometer of land area
population_age_60_69	Estimated population aged bewteen 60 to 69
population_age_70_79	Estimated population aged bewteen 70 to 79
population_age_80_and_older	Estimated population aged bewteen 80 and older
health_expenditure_usd	Government health expenditure per capita
smoking_prevalence	% of smokers in population
life_expectancy	Average years an individual is expected to live
nurses_per_1000	Nurses per 1000 people
region	Regional classification of the countries

Part 2: Graphical Analysis of the Data Set

Bar Chart

This first plot is a simple bar chart that describes each region and their Covid cases and Covid deaths count from 2020 till 2022. The aim of this plot is to show *which regions out of the 7 global regions were most affected by Covid 19 via deaths and infection.*

Since Covid deaths and cases are cumulative the max value for cumulative_confirmed and cumulative_deceased should represent the total deaths/cases of Covid for each any given country by the end of the year. Therefore, in order to produce this plot, the data is firstly grouped by year and country, then the max cumulative confirmed and deceased variables of each country were sumarised as the total confirmed/deceased values for the aggregated data.

```
cols_to_keep <- setdiff(names(covidDf), c("cumulative_confirmed", "cumulative_deceased", "date", "counti

aggregated_data_by_year <- covidDf %>%
    mutate(year = year(date)) %>%
    group_by(year, country_name) %>%
    summarise(total_confirmed = max(cumulative_confirmed),
              total_deceased = max(cumulative_deceased),
              across(all_of(cols_to_keep), first)
    )
```

```
## `summarise()` has grouped output by 'year'. You can override using the
## `.groups` argument.
```

Since there are over 200 different countries in the data set, in order to simplify for this plot the previous aggregated data is then grouped by the region of each country to find the total confirmed Covid cases and deaths of each region specified in the data set. The final data frame that will be used for the bar plot, `covid_deaths_n_cases` is shown below.

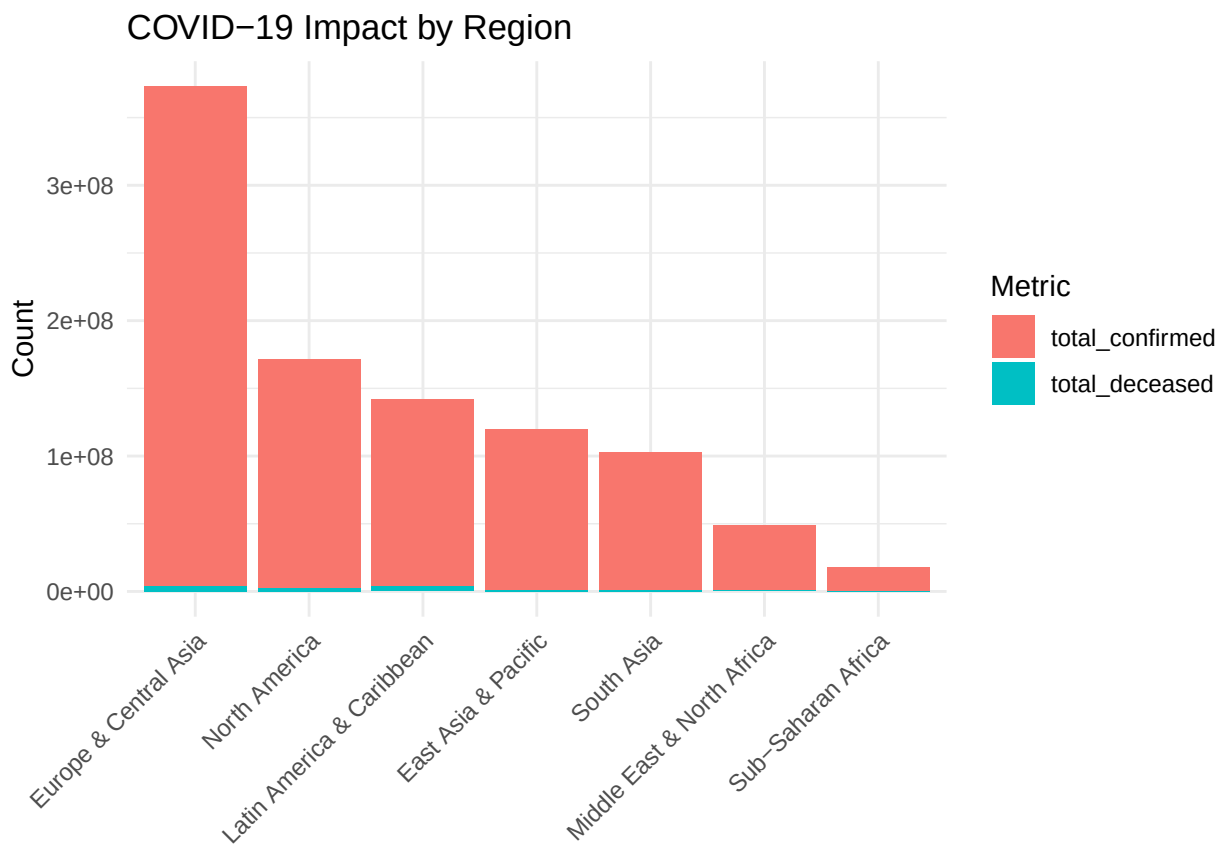
```
covid_deaths_n_cases <- aggregated_data_by_year %>%
  group_by(region) %>%
  summarise(
    total_confirmed = sum(total_confirmed),
    total_deceased = sum(total_deceased),
  )

covid_deaths_n_cases
```

```
## # A tibble: 7 x 3
##   region                      total_confirmed total_deceased
##   <fct>                                 <int>          <int>
## 1 East Asia & Pacific                118934766         866331
## 2 Europe & Central Asia              368643171        4339377
## 3 Latin America & Caribbean          138291556        3636563
## 4 Middle East & North Africa          48492472         727361
## 5 North America                      169708606        2221045
## 6 South Asia                         101346471        1369737
## 7 Sub-Saharan Africa                  17270068         361174
```

The plot is created using ggplot and the `geom_bar()` function as shown below. To create a stacked bar plot that incorporates both the total_deceased and total_confirmed statistics, the data frame is converted from a wide format to a long format using the `pivot_longer()` function from the tidyr package. The x axis containts the different regions and the y axis contains to total count of covid cases and deaths. The `reorder()` function arranges the columns in the bar chart in descending order of covid deaths and cases. Legends, axis labels and chart titles were added and x axis text was titled for more visual clarity.

```
covid_deaths_n_cases %>%
    pivot_longer(cols = c(total_confirmed, total_deceased),
                 names_to = "metric",
                 values_to = "count") %>%
    ggplot(aes(x = reorder(region, -count), y = count, fill = metric)) +
      geom_bar(stat = "identity") +
      labs(
        title = "COVID-19 Impact by Region",
        x = NULL,
        y = "Count",
        fill = "Metric"
      ) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```
covid_deaths_n_cases %>% ggplot(aes(x = reorder(region, - total_deceased), y = total_deceased))+
  geom_bar(stat = "identity") +
  labs(x = NULL, title = "COVID-19 Deaths by Region") +
```

```
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

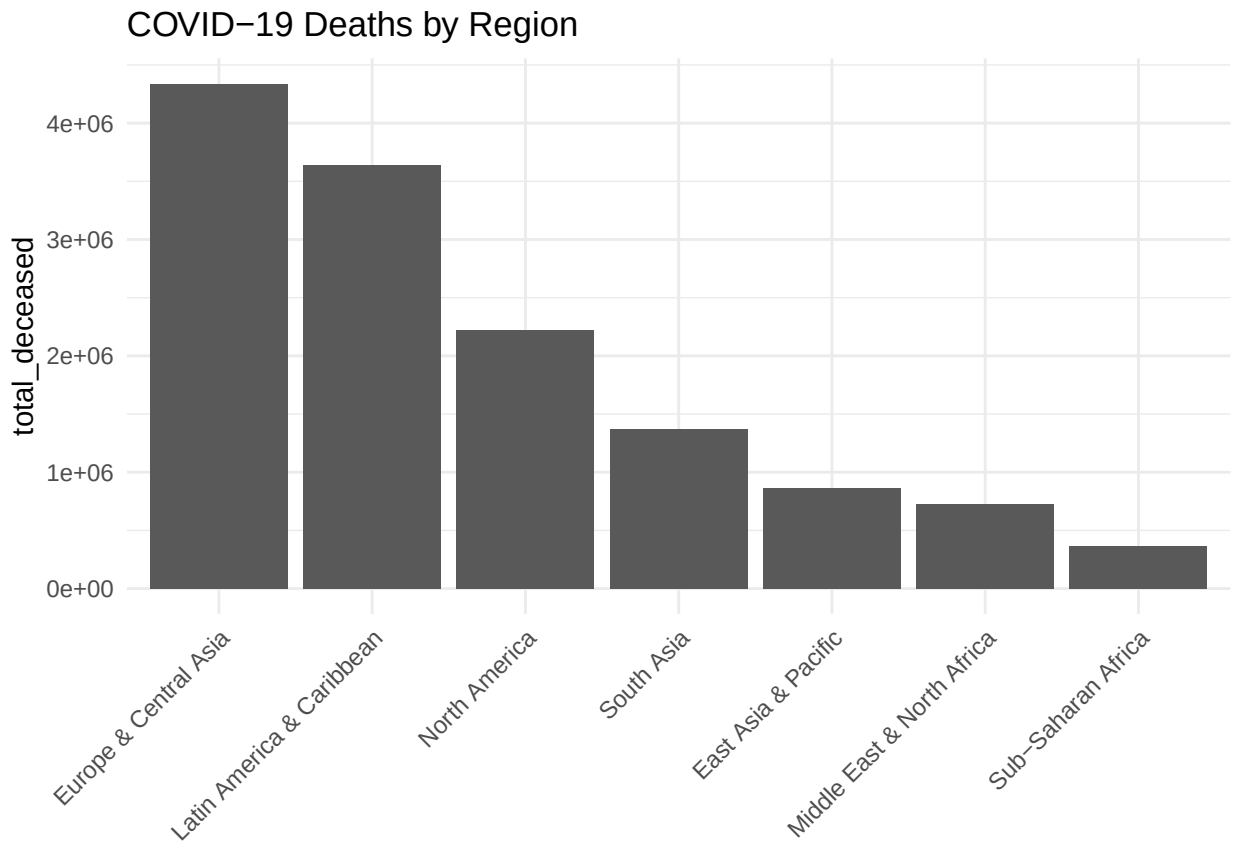


Figure 1.1: The figure contains 2 bar plots. The first showing the overall affect of Covid in terms of deaths and cases grouped by regions from 2020 to 2021 and another bar plot expanding on just Covid deaths for clearer visualisation. Both plots show that Covid cases and Covid deaths have a very linear relationship. The region most affected by Covid overall is Europe & Central Asia.

This plot is a simple but clear visualisation of which regions were most affected by Covid 19 from 2020 to 2022. In addition to showing which regions were most affected by Covid, as a preliminary plot to assess the data, Figure 1.1 also shows that although there are regions that have a more clear cut ratio of covid cases and covid deaths e.g., Europe & Central Asia and Sub-Saharan Africa which both respectively have the highest and lowest Covid cases and deaths count, in other words the most affected and least affected by Covid, there can be some deviation. Consider North America, which has the second highest Covid case/death count, is only third when looking just in terms of Covid deaths behind Latin America & Caribbean. This graphically shows that there is an inherent difference in these regions whether it be healthcare, population demographics or other factors that means more Covid cases does not necessarily equate to higher deaths from Covid.

Line Plot

This main aim of this part is to produce a simple line plot to show whether *there is a difference in spreading of Covid infection for urban and rural countries/regions?*.

Firstly, a `group_by()` function is used to aggregate the data on a regional level per each date recorded instead of country level as captured in the covidDf data set, this new data frame is labelled `aggregated_region_date`. Afterwards using the population_urban and population_rural variables, an urban to rural population ratio is calcuted. In addition to this, another variable called population_type is made under an ifelse condition, where if an observation has an urban_rural_ratio of less than 1, that region has a higher rural population than urban population, therefore it will be classified as "Rural Regions", otherwise the region will be classified as "Urban Regions" under the population type variable. The cases_per_100k variable was also calculated to standardise the differences in regional population as well as to more clearly interpret whether urban residents individually were at more risk.

```
aggregated_region_date <- covidDf %>% group_by(region, date) %>%
  summarise(cumulative_confirmed = sum(cumulative_confirmed),
            population = sum(population),
            population_rural = sum(population_rural),
            population_urban = sum(population_urban), .groups = "drop")

aggregated_region_date <- aggregated_region_date %>%
  mutate(
    urban_rural_ratio = (population_urban/population_rural),
    population_type = ifelse(urban_rural_ratio < 1, "Rural Regions", "Urban Regions"),
    cases_per_100k = (cumulative_confirmed/population)*100000
  )

#Changing population_type of a factor variable
aggregated_region_date$population_type <- factor(aggregated_region_date$population_type)
```

Using another `group_by()` function, the data frame is aggregated further by the region type i.e., an Urban Region or a Rural Region based on previously established population_type variable. Finally, the two plots are made using ggplot and the `geom_line()` function. The `color=population_type` and `group=population_type` parameters is used to produce and colour-code two different lines in the line plot for Urban regions and Rural Regions. The line size is set to 1.5 and labels are added for extra visual clarity. The two plots are then joined together using the pacthwork R package.

```
aggregated_poptype <- aggregated_region_date %>%
  group_by(population_type, date) %>%
  summarise(cumulative_confirmed = sum(cumulative_confirmed),
            cases_per_100k = sum(cases_per_100k))
```

```
## `summarise()` has grouped output by 'population_type'. You can override using
## the `.groups` argument.
```

```
f2p1 <- aggregated_poptype %>%
  ggplot(aes(x = date, y =  cumulative_confirmed, color=population_type, group=population_type)) +
  geom_line(size = 1.5) +
  labs(
    title = "Cumulative confirmed cases",
    x = "Date",
    y = "Confirmed Covid Cases",
    color = "Population Type",
    legend = NULL
  ) +
  theme_minimal()+
  theme(legend.position = "none")
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```r
f2p2 <- aggregated_poptype %>%
  ggplot(aes(x = date, y =  cases_per_100k, color=population_type, group=population_type)) +
  geom_line(size = 1.5) +
  labs(
    title = "Cases per 100k population",
    x = "Date",
    y = "Confirmed Covid Cases per 100k people",
    color = "Population Type"
  ) +
  theme_minimal()

library(patchwork)
f2p1 + f2p2
```

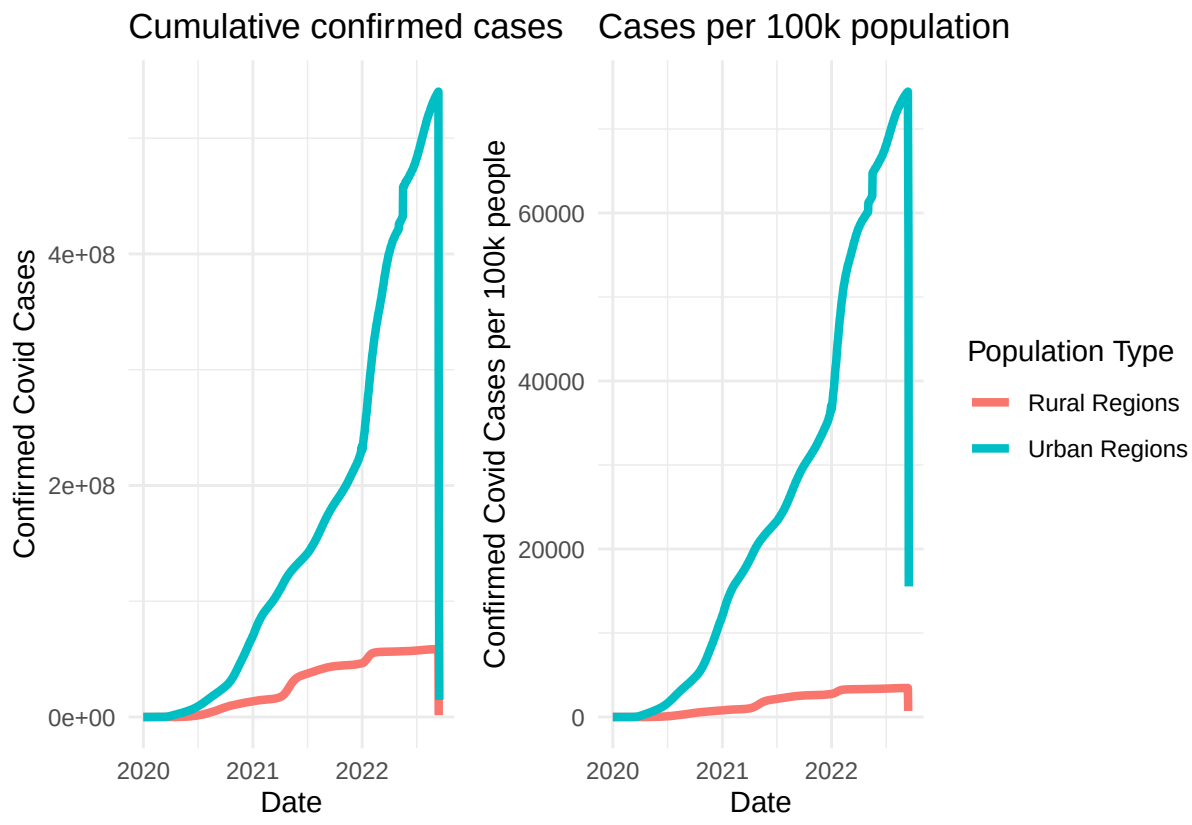


Figure 1.2: Line plots showing the number of confirmed Covid cases/cases per 100k people overtime (2020-2022) grouped by rural regions and urban regions i.e., regions that has a larger rural population compared to urban population and vice versa. It can be seen that from start to end regions with higher urban populations consistently have higher Covid cases compared to more rural populated regions.

As observed from Figure 1.2, both raw cumulative cases and cases per 100k people overtime shows nearly the exact same trend. The Urban Regions line is consistently higher and steeper, especially from 2021 onward

showing that regions with a higher urban population are associated with higher Covid cases as well as faster transmission of Covid as shown by the higher growth rate. On the contrary, the red line representing regions with a higher rural population experiences significantly slow growth and plateaus at a lower level, either indicating fewer outbreaks and less Covid impact; the probability of this difference is primarily due to unreported cases in rural regions is possible, but because of how large the difference is, it is unlikely. Overall, the plot shows a strong association between urbanisation and higher Covid case counts.

Violin Plot

For this part of the investigation, the primary question is - *is there a difference in Covid death numbers for countries with older populations vs countries with younger populations?*

To answer this, first the data needs to be aggregated further by country name instead of just year so that values like total_confirmed and total_deceased represents the total cases and deaths across 2020 to 2022. This turns the data from a time series to a cross sectional data set, this is important to do as the values for population are fixed due to the limitations of the data.

```
aggregated_data <- aggregated_data_by_year %>% group_by(country_name) %>%
    summarise(total_confirmed = sum(total_confirmed),
              total_deceased = sum(total_deceased),
              across(all_of(cols_to_keep), first))
```

For this analysis, people age 60 and above will be considered elderly and potentially of higher risks from Covid. Another variable, `elderly_pop_proportion` is calculated to capture the proportion of a country's population that are considered elderly.

```
aggregated_data <- aggregated_data %>%
  mutate(elderly_pop_proportion = 100*(population_age_60_69 + population_age_70_79 + population_age_80_a


summary(aggregated_data$elderly_pop_proportion)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   3.145   5.828  11.172  13.730  21.799  36.201
```

The summary statistics above shows that if a country has ~21.8% or more of their population being elderly individuals then that country is within the top 75% for this metric. Using this information, rounding 21.8 down to 20%, countries with an elderly population proportion higher than 20% will be considered to have an "Older Population" whereas 20% or less will be considered "Younger Population" countries. These conditions are captured in the factor variable `age_group` created using the code and the `ifelse` logic shown below. Additionally, using raw total Covid death count could be misleading as countries with a larger population with naturally higher death counts regardless of age structure could inflate the values for this analysis. Therefore to adjust for population size, the plot will use death counts per 100k people. The death_per_100k variable is also log transformed to account for skewness in the variable.

```
aggregated_data <- aggregated_data %>%
  mutate(
    age_group = ifelse(elderly_pop_proportion > 20, "Older Population", "Younger Population"),
    death_per_100k = (total_deceased/population) * 100000,
    log10_death_per_100k = log10(((total_deceased/population) * 100000)+1)
  )
aggregated_data$age_group <- factor(aggregated_data$age_group)
```

Using ggplot in combination with the `geom_violin` and `geom_boxplot` functions a violin plot is produced to showcase the distribution of Covid deaths for younger and older population countries.

```
ggplot(aggregated_data, aes(x = age_group, y = log10_death_per_100k, fill = age_group)) +
  geom_violin() +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(
    x = NULL,
    y = "Log10(Total Covid Deaths per 100k + 1)",
    title = "Distribution of Covid Deaths per 100k people by Age Group"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

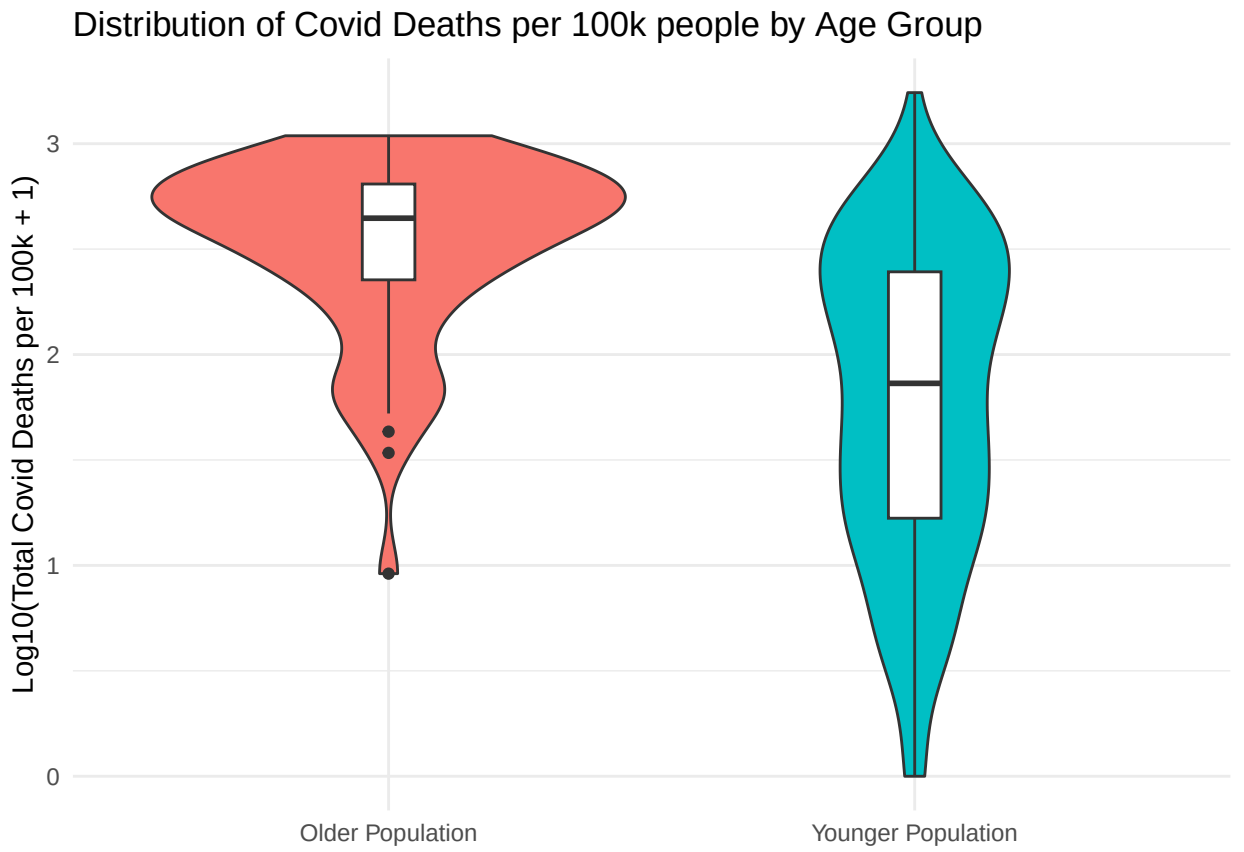


Figure 1.3: Violin plot with layered box plots comparing the distribution of Covid-19 deaths from countries with younger vs. countries with older populations across 2020 to 2022. The death counts are standardised per 100k people and log transformed to provide a clearer comparison between older and younger populations. The curvature of the violin plot shows the density of countries at different 'death levels' and the white boxplots inside shows the median and interquartile range (IQR). It is seen that countries with older population proportion have higher median death counts and lower spread.

Shown in figure 1.3, both violins shows long tails which indicating some countries had very low death counts compared to the average. The lower spread for countries with older populations could suggest that these countries experienced more similar levels of Covid impact compared to nations with younger populations; this could also be due to the lower sample size. Overall, there is a visible difference between the distribution of Covid deaths in the older and younger populations. There appears to be a higher concentration of countries with high elderly population proportion at high death levels comparatively.

Confidence intervals (CI) for both populations of countries could also be used to further test these claims.

The formula for a confidence interval of a normally distributed random variable is as follows:

$$CI = \bar{x} \pm z \cdot \frac{s}{\sqrt{n}}$$

For this analysis, a 95% CI will be used where the z value can be found using the function `qnorm(1-(0.05/2))`. Using the formula above, the code below calculates all of the necessary values and produces the CIs for older and younger population countries. Afterwards the CI is converted back into a standard death scale instead of log10 for easier interpretation.

```
older_pop <- aggregated_data %>% filter(age_group == "Older Population")

nO <- length(older_pop$log10_death_per_100k)
muO <- mean(older_pop$log10_death_per_100k)
z <- qnorm(1-(0.05/2))
sO <- sd(older_pop$log10_death_per_100k)
uprCI_O <- muO + z * (sO/sqrt(nO))
lwrCI_O <- muO - z * (sO/sqrt(nO))
CI_O_log <- c(lwrCI_O, uprCI_O)
CI_O_standard <- 10^CI_O_log - 1

younger_pop <- aggregated_data %>% filter(age_group == "Younger Population")

nY <- length(younger_pop$log10_death_per_100k)
muY <- mean(younger_pop$log10_death_per_100k)
sY <- sd(younger_pop$log10_death_per_100k)
uprCI_Y <- muY + z * (sY/sqrt(nY))
lwrCI_Y <- muY - z * (sY/sqrt(nY))
CI_Y_log <- c(lwrCI_Y, uprCI_Y)
CI_Y_standard <- 10^CI_Y_log - 1 #10^CI - 1 undoes the log10 transformation

#Putting the CIs into a tibble
tibble(
  Variable = c("Older Population", "Younger Population"),
  `Lower Bound` = c(CI_O_standard[1], CI_Y_standard[1]),
  `Upper Bound` = c(CI_O_standard[2], CI_Y_standard[2]),
  `Mean Value` = 10^c(muO, muY) - 1
)
```

```
## # A tibble: 2 x 4
##   Variable           `Lower Bound` `Upper Bound` `Mean Value`
##   <chr>                      <dbl>         <dbl>        <dbl>
## 1 Older Population           255.          420.         327.
## 2 Younger Population          44.1          76.4         58.1
```

With these results we are 95% confident that the true population mean Covid death count per 100K people between 2020 and 2022 of countries with an older population is between 255 and 420 deaths. For countries with a younger population we are 95% confident that the true mean Covid death count is between 44 and 76 deaths.

Due to the lower sample size the range both confidence intervals could be considered large.. There is overall higher variability especially for the sample of countries with older population. However, the results show that both of the intervals do not overlap i.e., the upper bound for the younger population interval (76

deaths) sits below the lower bound for the older population interval (255 deaths). This shows that there is a significant statistical difference for the distribution Covid death per 100k people between countries with older and younger population proportions.

This could also be further verified using a two sample t-test shown with code below

```
t.test(older_pop$log10_death_per_100k, younger_pop$log10_death_per_100k, alternative = "two.sided")
```

```
##
##  Welch Two Sample t-test
##
## data:  older_pop$log10_death_per_100k and younger_pop$log10_death_per_100k
## t = 9.1638, df = 183.14, p-value < 2.2e-16
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.5844523 0.9051737
## sample estimates:
## mean of x mean of y
##  2.516337  1.771524
```

The results above shows a p value of less than 2.2x10^-16. This can be interpreted as; we are 99% confident that there is a statistical difference in the population mean Covid deaths of countries with older population proportions and countries with a younger population.

The two sample t-test 95% CI is (0.584, 0.905) in log10 scale; this needs to be interpreted a little differently than before in order to check the statistical significance in the difference of the two populations. Consider the operation of subtracting logs:

$$log(X_O) - log(X_Y) = log(\frac{X_O}{X_Y})$$

Therefore, if the values for Covid deaths were originally log transformed, then the back transformation to standard death units would give a ratio between the two populations instead. So the CI is still calculated as follows:

```
t_uprCI <- 10^0.9051737-1
t_lwrCI <- 10^0.5844523-1
c(t_lwrCI, t_uprCI)
```

```
## [1] 2.841071 7.038476
```

However it must be interpreted as ; we are 95% confident that the true mean Covid death count per 100k people of countries with older populations is between 2.841 to 7.038 times the countries with a younger population proportion.

Scatter Plot

To establish a relationship between a countries' smoking prevalence and the risk of death from Covid-19 a scatter plot is produced. The aim of this plot is to answer whether *does more countries with more smoking prevalance have a higher likelihood of worse Covid death counts?*. This will be done by analysing the created plot and comparing the number of Covid deaths from 2020 to 2022 of a country against a countries' smoking prevalence

Similar to the previous violin plot, using the raw total covid death count potentially could lead to misleading visualisations of the data since it does not account for the fact that higher populations would mean naturally higher death counts from Covid. Therefore this visualisation will also use the variable `death_per_100k`

The graph is made using ggplot and the `geom_point()` function. `death_per_100k` is the y axis and population_density for the x axis, additionally, the `size = gdp_per_capita_usd` parameter maps the size of each data point to its respect gdp per capita in the aggregated_data data frame for further analysis of these variable's interactions. Each data point is also colour coded by their respective region and the `scale_size()` function scales the potential sizes of the data points from a value of 2 to 10 for a clearer plot.

```
aggregated_data %>%
  ggplot(aes(x = smoking_prevalence,
                                y = death_per_100k,
                                size = gdp_per_capita_usd,
                                color = region)) +
  geom_point() +
  scale_size(range = c(2, 10)) +
  labs(
    title = "Covid-19 Cases per 100k vs Smoking Prevalence",
    x = "Smokers as a % of population",
    y = "Covid deaths per 100k people"
  )
```

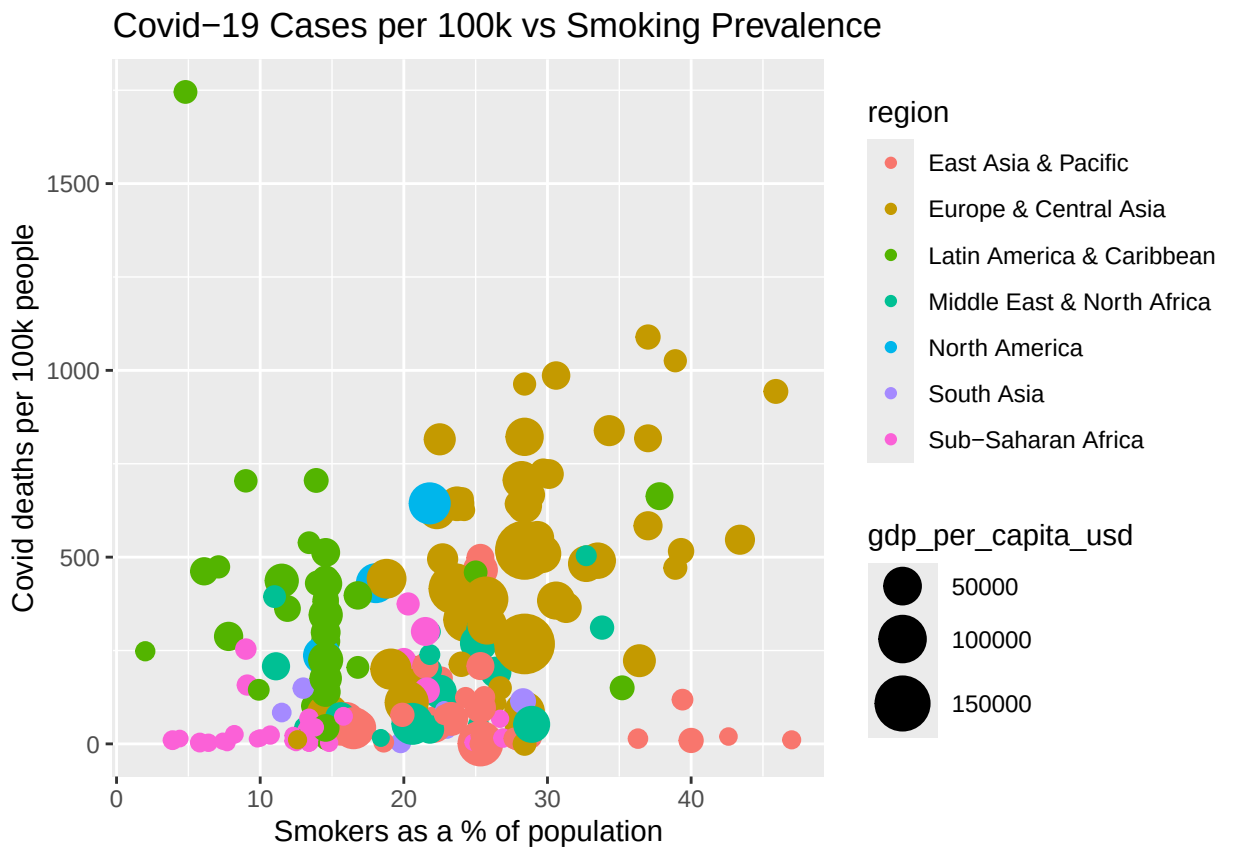


Figure 1.4: Scatter/bubble plot showing the relationship between Covid deaths per 100k people and the countries' smoking prevalence. Each bubble represents one country with the bubble sizes representing the GDP per capita in USD of each country, the countries are also colour coded with their respective region. The plot shows that there is not a very clear relationships between Covid deaths and smoking, there are many countries with similar smoking prevalence but with widely varying levels of confirmed Covid cases.

Figure 1.4 shows that smoking prevalence by itself does is not a strong explanatory factor for Covid death rates and that other variables likely play a much larger role. With closer inspection, countries in Europe & Central Asia specifically does show a moderate postive linear relationship between smoking prevalence and death counts, but the same cannot be confidently said for the other regions. Other interesting observations from the plot includes visible regional clustering for smoking prevalence e.g., Europe & Central Asia is primarily concentrated around 30-40% of smokers in population. South Asia & Sub-Saharan Africa are grouped closer to the front with a lower population of smokers. It is also interesting to note that the GDP per capita for each country when distributed by smoking prevalence forms almost a bell-curve-like pattern. Countries with generally smaller GDP either have a relatively low smoking prevalence (15%<), or a higher smoking prevalence (>35%), countries that appears to have the highest GDPs sit in the middle around 20-30%.

Heatmap

The aim of this plot will be to *show the correlation between all numerical variables within the covidDf data set.* This is done by using a heatmap to show the linear relationships strengths of the different variables using the reshape2 and scales R packages.

The code below selects all of the numerical variables excluding the population age categories variables to improve the plot's clarity as they are primarily represented within the elderly_pop_proportion variable. The `cor()` function is used to generate the correlation coefficients for all of the variable combinations, these are stored in the cordata variable.

```
pacman::p_load(reshape2, scales) #Loading in the required packages

heatmap_df <- aggregated_data %>%
  select(2:16, elderly_pop_proportion, -population_age_60_69, -population_age_70_79, -population_age_80_

cordata <- cor(heatmap_df)
```

The `melt()` function below takes the correlation data and reshapes it into a long data format that is suitable for heatmaps. In essence, the `melt()` function creates a data frame with 3 columns; Var1, Var2 and value. The names of the data set's variables are stored in Var 1 and Var2 and the correlation coefficients of the corresponding variables is contained in the value column. The values are also rounded to 2 digits.

```
heatmap_data <- melt(cordata)
heatmap_data <- heatmap_data %>%
  mutate(across(where(is.numeric), ~  round(., digits = 2)))
```

Using ggplot and the `geom_tile()` function the heatmap is created by plotting Var1 of the heatmap_data on the x axis and Var2 on the y axis. The gradient scale is added to help visualise the strength of each linear relationship better e.g., stronger relationships with coefficients closer to a value of 1 has a darker red colour whilst weaker relationships will appear a lighter yellow.

```
heatmap_data %>% ggplot(aes(x = Var1, y = Var2, fill = value, label = value)) +
  geom_tile() +
  geom_text(size = 2.5) +
  labs(title = "Correlation Heatmap of Covid 2020-2022 Statistics",
       x = NULL, y = NULL) +
  scale_fill_gradient(low = "lightyellow", high = "red") +
```

```
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 60, hjust = 1),
      axis.text.y = element_text(size = 8))
```

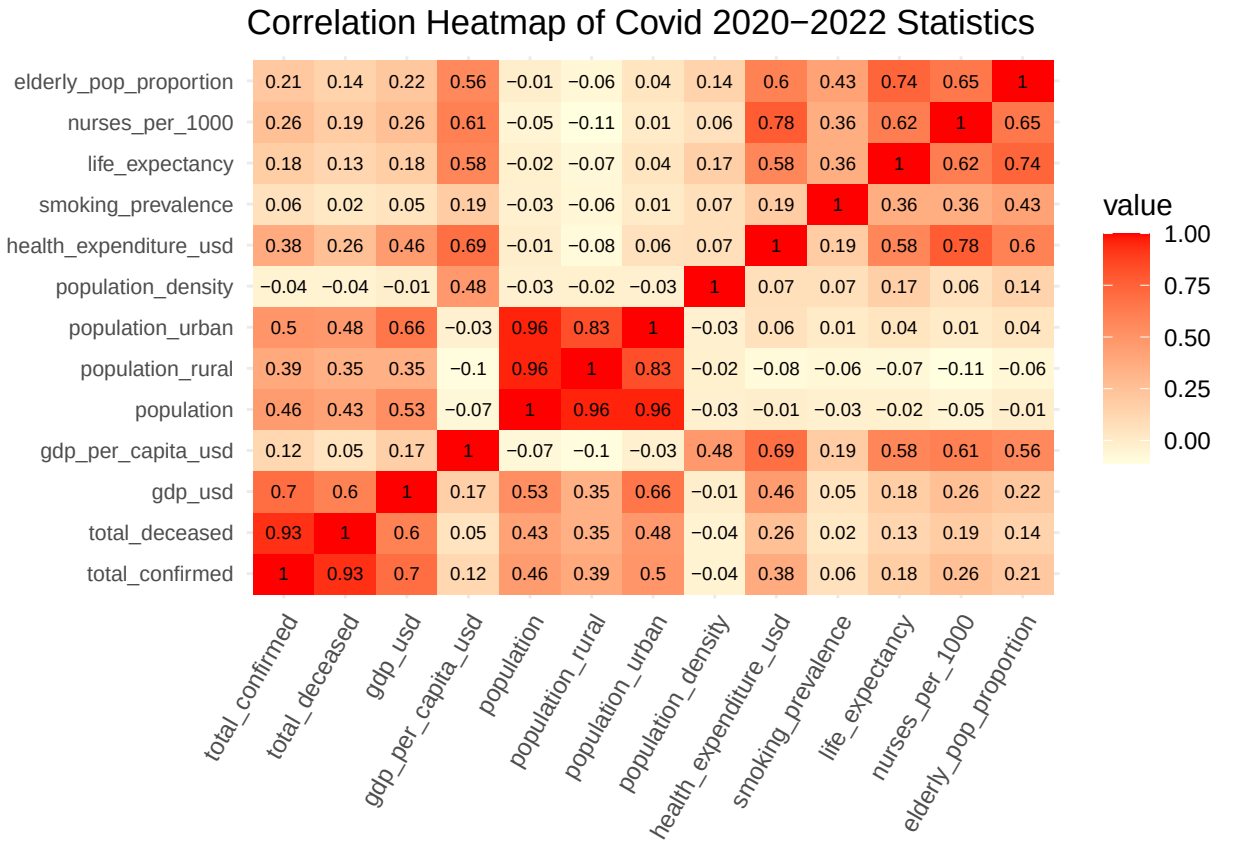


Figure 1.5: The heatmap shows the correlation between numeric variables within the covidDf dataset. The darker the red colour on the tiles the higher the correlation between the two corresponding variables. For example, the most highly correlated variables are total_deceased and total_confirmed, which makes sense as you would generally expect, higher total cases of Covid would naturally mean higher deaths from Covid.

Shown in Figure 1.5, a majority of the variables are weakly positvely correlated with each other. There are some strongly correlated variables for obvious reasons such as total_deceased & total_confirmed or life_expectancy and elderly_pop_proportion or even government health expenditures and nurses_per_1000. Otherwise there are also some interesting relationships to note. For example, total_deceased and total_confirmed both have a moderate positive correlation of 0.6 and 0.7 respectively with GDP of the country yet both also have a weak relationship with GDP_per_capita. Breaking it down, this does logically make sense as normal GDP measures the absolute size of the economy, countries with bigger populations have larger GDPs but also is more prone to higher cases of Covid, on the other hand GDP per capita accounts for the population size which then shows more clearly that the relationship between Covid deaths and cases and GDP are not very strong.

Part 3: Applying Predictive Methods to Find Correlations and Trends in the Data

This part of the investigation will apply two different predictive methods, multiple linear regression (MLR) and random forests to the Covid data in order to identify different trends and correlations as well as to see which predictive method performs better. For simplicity, the `aggregated_data` data frame created previously will be used for this analysis as it combines the entire covidDf panel data set into a cross sectional data set so time as a variable will not be considered.

Multiple Linear Regression

For MLR the target variable will be the imputed value of cases_per_100k - covid cases per 100k people, all of the predictor variables of interest are log_cases_per_100k, log_gdp_per_capita, log_health_expenditure, nurses_per_1000, population_urban, elderly_pop_proportion, population_density, life_expectancy, smoking_prevalence, region. Cases_per_100k, health_expenditure_usd and gdp_per_capita_usd shown in Part 2 analysis and generally speaking are variables that are expected to have a stronger skewed distribution, to handle this skewness these variables are log transformed with log base 10.

```
pacman::p_load(tidymodels, car)

aggregated_data <- aggregated_data %>% mutate(
  cases_per_100k = (total_confirmed/population)*100000,
  log_cases_per_100k = log10(cases_per_100k+ 1),
  log_health_expenditure = log10(health_expenditure_usd + 1),
  log_gdp_per_capita = log10(gdp_per_capita_usd + 1)
)
```

All of the variables mentioned previously that are being used for this MLR analysis is combined to make the predictiondata data frame which will be used as the prediction dataset for predicting log transformed cases per 100k people. Using the tidymodels R package, the prediction data is then randomly split; 20% into a testing set and 80% into a training set. In summary, the training set covid_train with 158 observations will be used to develop the MLR model, then the model will be tested against the covid_test data of 53 observations.

```
set.seed(111)
mlr_predictiondata <- aggregated_data %>%
  select(log_cases_per_100k, log_gdp_per_capita,
         log_health_expenditure,
         nurses_per_1000, population_urban,
         elderly_pop_proportion, population_density,
         life_expectancy, smoking_prevalence,
         region)

mlr_split <- initial_split(mlr_predictiondata)
mlr_split
```

```
## <Training/Testing/Total>
## <158/53/211>
```

```
mlr_train <- training(mlr_split)
mlr_test <- testing(mlr_split)
```

The `recipe()` function from the car package is used to create a pre-processing blueprint for the data before modeling, this is named covid_recipe. In this case, it is primarily used to turn the categorical variable of region into dummy variables.

```
mlr_recipe <- recipe(log_cases_per_100k ~., data = mlr_train) %>%
  step_dummy(all_nominal(), - all_outcomes())  %>%
  prep()
```

The juice and bake functions applies the 'recipe' to the training and testing datasets previously specified.

```
mlr_train_preproc <- juice(mlr_recipe)
mlr_test_preproc <- bake(mlr_recipe, mlr_test)
```

Model Selection: Backwards Stepwise Selection

When developing any prediction models, exclusion of important terms clearly leads to an incorrect model, which causes incorrect or misleading conclusions. However, including unnecessary terms diminishes the value of the model as a simplification of the data, and also reduces the statistical accuracy of predictions. This highlights the importance of model selection and as a part of this process the backwards stepwise selection method is used in order to filter out the more statistically insignificant variables.

The `Anova()` function tests whether each predictor variable significantly contributes to predicting to response variable individually whilst holding all other predictors constant. It returns the F-statistics and p-values for each variable's contirbution. Using this, starting with a function that includes all of the variables of interest, the variable that is the most statistically insignificant - highest p-value over 0.05 will be removed, the function is tested again and this is repeated until all variables remain statistically significant (p-value less than 0.05).
Step 1

Here the interaction terms:

- log_gdp_per_capita:log_health_expenditure
- population_urban:population_density
- elderly_pop_proportion:life_expectancy

are added as these variables are generally expected to interact with each other and have a combined affect on predicting the response variable. x

```
covid_lm0 <- lm(log_cases_per_100k~. + log_gdp_per_capita*log_health_expenditure
                + population_density*population_urban
                + life_expectancy*elderly_pop_proportion,
                data = mlr_train_preproc)

Anova(covid_lm0)
```

```
## Anova Table (Type II tests)
##
## Response: log_cases_per_100k
##                                    Sum Sq  Df F value   Pr(>F)
## log_gdp_per_capita                  1.086   1  2.8352 0.094447 .
## log_health_expenditure              0.270   1  0.7037 0.402964
## nurses_per_1000                     0.008   1  0.0197 0.888495
```

```
## population_urban                              2.155   1  5.6264 0.019053 *
## elderly_pop_proportion                        0.265   1  0.6920 0.406886
## population_density                            2.432   1  6.3479 0.012876 *
## life_expectancy                               4.210   1 10.9892 0.001167 **
## smoking_prevalence                            1.170   1  3.0550 0.082682 .
## region_Europe...Central.Asia                  0.112   1  0.2927 0.589384
## region_Latin.America...Caribbean              1.296   1  3.3820 0.068032 .
## region_Middle.East...North.Africa             0.188   1  0.4915 0.484434
## region_North.America                          0.069   1  0.1800 0.672010
## region_South.Asia                             1.135   1  2.9617 0.087467 .
## region_Sub.Saharan.Africa                     2.528   1  6.5987 0.011253 *
## log_gdp_per_capita:log_health_expenditure     0.026   1  0.0671 0.795978
## population_urban:population_density           0.158   1  0.4119 0.522049
## elderly_pop_proportion:life_expectancy        1.133   1  2.9574 0.087697 .
## Residuals                                    53.634 140
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

nurses_per_1000 had the highest p-value of 0.888; therefore it is removed in the next step.

```
covid_lm1 <- lm(log_cases_per_100k~. -nurses_per_1000
                + log_gdp_per_capita*log_health_expenditure
                + population_density*population_urban
                + life_expectancy*elderly_pop_proportion,
                data = mlr_train_preproc)

Anova(covid_lm1)
```

Step 2

```
## Anova Table (Type II tests)
##
## Response: log_cases_per_100k
##                                              Sum Sq  Df F value   Pr(>F)
## log_gdp_per_capita                            1.068   1  2.8071 0.096066 .
## log_health_expenditure                        0.247   1  0.6497 0.421574
## population_urban                              2.157   1  5.6705 0.018588 *
## elderly_pop_proportion                        0.272   1  0.7148 0.399287
## population_density                            2.457   1  6.4588 0.012122 *
## life_expectancy                               4.229   1 11.1154 0.001094 **
## smoking_prevalence                            1.196   1  3.1427 0.078426 .
## region_Europe...Central.Asia                  0.105   1  0.2749 0.600912
## region_Latin.America...Caribbean              1.376   1  3.6174 0.059217 .
## region_Middle.East...North.Africa             0.209   1  0.5487 0.460098
## region_North.America                          0.072   1  0.1881 0.665175
## region_South.Asia                             1.154   1  3.0347 0.083684 .
## region_Sub.Saharan.Africa                     2.624   1  6.8984 0.009581 **
## log_gdp_per_capita:log_health_expenditure     0.041   1  0.1067 0.744439
## population_urban:population_density           0.155   1  0.4071 0.524461
## elderly_pop_proportion:life_expectancy        1.126   1  2.9590 0.087597 .
## Residuals                                    53.641 141
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

All of the interactino terms were all statistically insignificant, they are all removed for this step.

```
covid_lm2 <- lm(log_cases_per_100k~. -nurses_per_1000,
                data = mlr_train_preproc)

Anova(covid_lm2)
```

Step 3

```
## Anova Table (Type II tests)
##
## Response: log_cases_per_100k
##                                  Sum Sq  Df F value   Pr(>F)
## log_gdp_per_capita                1.041   1  2.6840 0.103544
## log_health_expenditure            0.248   1  0.6393 0.425276
## population_urban                  2.019   1  5.2042 0.023999 *
## elderly_pop_proportion            0.186   1  0.4794 0.489800
## population_density                2.675   1  6.8939 0.009584 **
## life_expectancy                   3.813   1  9.8257 0.002086 **
## smoking_prevalence                1.911   1  4.9244 0.028044 *
## region_Europe...Central.Asia      0.135   1  0.3471 0.556660
## region_Latin.America...Caribbean  2.668   1  6.8750 0.009681 **
## region_Middle.East...North.Africa 0.348   1  0.8957 0.345514
## region_North.America              0.130   1  0.3345 0.563923
## region_South.Asia                 0.972   1  2.5043 0.115729
## region_Sub.Saharan.Africa         1.688   1  4.3503 0.038767 *
## Residuals                        55.876 144
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Although some of the dummy variables for region is insignificant since at least 1 dummy variable for the region variable is significant it is kept. Therefore, `elderly_pop_proportion` is the next highest p-value of 0.489 and therefore is removed.

```
covid_lm3 <- lm(log_cases_per_100k~. -nurses_per_1000 -elderly_pop_proportion,
                data = mlr_train_preproc)

Anova(covid_lm3)
```

Step 4

```
## Anova Table (Type II tests)
##
## Response: log_cases_per_100k
```

```
##                                   Sum Sq  Df F value    Pr(>F)
## log_gdp_per_capita                 1.253   1  3.2418 0.0738595 .
## log_health_expenditure             0.280   1  0.7236 0.3963624
## population_urban                   1.927   1  4.9838 0.0271179 *
## population_density                 2.759   1  7.1349 0.0084241 **
## life_expectancy                    5.073   1 13.1197 0.0004033 ***
## smoking_prevalence                 2.234   1  5.7792 0.0174796 *
## region_Europe...Central.Asia       0.313   1  0.8104 0.3695087
## region_Latin.America...Caribbean   2.849   1  7.3676 0.0074467 **
## region_Middle.East...North.Africa  0.248   1  0.6405 0.4248516
## region_North.America               0.080   1  0.2064 0.6502939
## region_South.Asia                  1.038   1  2.6859 0.1034058
## region_Sub.Saharan.Africa          1.903   1  4.9229 0.0280573 *
## Residuals                         56.062 145
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

`log_health_expenditure` has the highest p-value of 0.396

```
covid_lm4 <- lm(log_cases_per_100k~. -nurses_per_1000 -elderly_pop_proportion
                -log_health_expenditure,
                data = mlr_train_preproc)

Anova(covid_lm4)
```

Step 5

```
## Anova Table (Type II tests)
##
## Response: log_cases_per_100k
##                                   Sum Sq  Df F value    Pr(>F)
## log_gdp_per_capita                 5.658   1 14.6619 0.0001903 ***
## population_urban                   1.983   1  5.1387 0.0248676 *
## population_density                 3.395   1  8.7981 0.0035240 **
## life_expectancy                    6.280   1 16.2731 8.809e-05 ***
## smoking_prevalence                 2.406   1  6.2337 0.0136448 *
## region_Europe...Central.Asia       0.384   1  0.9941 0.3203895
## region_Latin.America...Caribbean   2.762   1  7.1574 0.0083182 **
## region_Middle.East...North.Africa  0.242   1  0.6265 0.4299271
## region_North.America               0.036   1  0.0941 0.7594976
## region_South.Asia                  0.894   1  2.3170 0.1301305
## region_Sub.Saharan.Africa          1.904   1  4.9334 0.0278817 *
## Residuals                         56.342 146
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Finally, since the investigation uses a 5% significance level, all of the variables now are statistically significant with p-values under 0.05. Therefore, covid_lm4 will represent the final MLR model for the dataset.

```r
summary(covid_lm4)
```

```
##
## Call:
## lm(formula = log_cases_per_100k ~ . - nurses_per_1000 - elderly_pop_proportion -
##     log_health_expenditure, data = mlr_train_preproc)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.8362 -0.1801  0.0813  0.2529  1.2710
##
## Coefficients:
##                                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)                     -3.032e+00  8.663e-01  -3.500 0.000617 ***
## log_gdp_per_capita               5.384e-01  1.406e-01   3.829 0.000190 ***
## population_urban                -1.607e-09  7.089e-10  -2.267 0.024868 *
## population_density              -5.674e-05  1.913e-05  -2.966 0.003524 **
## life_expectancy                  5.749e-02  1.425e-02   4.034 8.81e-05 ***
## smoking_prevalence               2.021e-02  8.093e-03   2.497 0.013645 *
## region_Europe...Central.Asia     1.691e-01  1.696e-01   0.997 0.320389
## region_Latin.America...Caribbean 5.246e-01  1.961e-01   2.675 0.008318 **
## region_Middle.East...North.Africa 1.566e-01  1.979e-01   0.792 0.429927
## region_North.America            -1.451e-01  4.732e-01  -0.307 0.759498
## region_South.Asia                4.510e-01  2.963e-01   1.522 0.130131
## region_Sub.Saharan.Africa        5.096e-01  2.294e-01   2.221 0.027882 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6212 on 146 degrees of freedom
## Multiple R-squared:  0.5923, Adjusted R-squared:  0.5615
## F-statistic: 19.28 on 11 and 146 DF,  p-value: < 2.2e-16
```

The final linear regression model is:

log_cases_per_100k = -3.032 + 0.5384 * log_gdp_per_capita - 1.607e-9 * population_urban - 5.674e-5 * population_density + 0.05749 * life_expectancy + 0.02021 * smoking_prevalence + 0.1691 * region_Europe_Central_Asia + 0.5246 * region_Latin_America_Caribbean + 0.1566 * region_Middle_East_North_Africa - 0.1451 * region_North_America + 0.4510 * region_South_Asia + 0.5096 * region_Sub_Saharan_Africa

Where the `region_*` variables are dummy indicators (1 if the country is in that region, 0 otherwise).

Model Selection: Analysis and Checking Assumptions

The code below applies the covid_lm4 model to the previously specified testing data. The results are documented within the predictedvsactual tibble shown below; it contains both the log transformed values and the back transformed values for those log values for easier interpretations.

```r
predicted_values <- predict(covid_lm4, newdata = mlr_test_preproc)
predictedvsactual <- tibble(
  `Log Predicted Values` = c(predicted_values),
  `Log Actual Values` = c(mlr_test_preproc$log_cases_per_100k),
  `Predicted Values` = c(10^predicted_values - 1),
```

```
  `Actual Values` = c(10^(mlr_test_preproc$log_cases_per_100k) - 1)
)

predictedvsactual
```

```
## # A tibble: 53 x 4
##    `Log Predicted Values` `Log Actual Values` `Predicted Values` `Actual Values`
##                     <dbl>               <dbl>              <dbl>           <dbl>
##  1                   3.90                4.50              7884.          31861.
##  2                   4.96                4.88             91194.          75007.
##  3                   4.45                4.66             28266.          45646.
##  4                   3.98                4.30              9657.          20009.
##  5                   4.05                4.45             11241.          28225.
##  6                   2.79                2.66               612.            461.
##  7                   3.78                4.20              6028.          15989.
##  8                   2.22                2.84               164.            689.
##  9                   5.08                4.60            119252.          39740.
## 10                   4.53                4.56             33588.          36271.
## # i 43 more rows
```

This investigation will use the R^2 value and the root mean squared error (RMSE) as metrics that measures a model's performance. The R^2 for linear models measures how much variance in the outcome can be explained by the model's prediction e.g., 1 meaning the model can explain 100% of the variation in the data whereas 0 means no predictive power; the closer to 1 the better. The R^2 value the covid_lm4 MLR model is 0.5923, given by the summary() function used above, meaning that the model explains around ~59.23% of the variance in the testing data.

As for RMSE, it measures the difference between values predicted by the model and the actual values within the dataset. Unlike R^2 which describes the model's goodness of fit, RMSE shows the magnitude of which the predictions are accurate. It is calculated using the code below.

```
rmselm <- sqrt(mean((predictedvsactual$`Actual Values` -
                       predictedvsactual$`Predicted Values`)^2))
rmselm
```

```
## [1] 21328.82
```

The model's RMSE is 21328.82 (covid cases per 100k people)

Using ggplot in combination with the `geom_point()` function a predicted vs actual scatter plot is created. A red dashed 45 degree line or the 'line of perfect prediction' is added using the `geom_abline()` function and the line of best fit is added using the `geom_smooth()` function.

```
predictedvsac_plot <-
  predictedvsactual %>% ggplot(aes(x = `Actual Values`, y = `Predicted Values`)) +
    geom_point() +
    geom_smooth(method = "lm") +
    geom_abline(intercept=0, slope = 1, linetype="dashed", color = "red") +
    labs(title = "Predicted vs Actual Values - MLR") +
    theme_minimal()

predictedvsac_plot
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

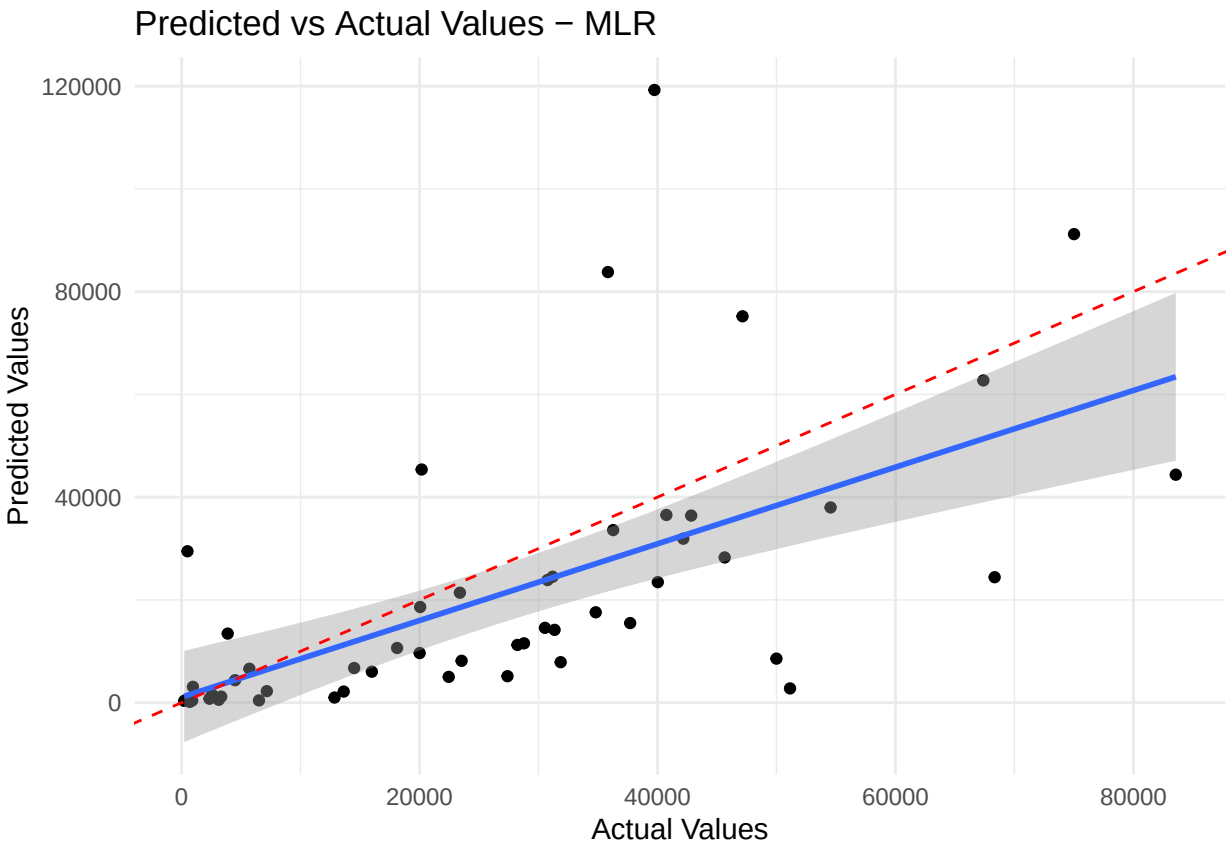


Figure 2.1: Scatter plot showing the predicted vs actual values from the covid_lm4 model. The predicted values are on the y-axis, actual values on x-axis. The red dotted line represents the trend line for an ideal model with perfect predictions, the blue line represent the model's actual trend line. The plot clearly shows that the trend line are consistently below the line of perfect prediction.

Shown in Figure 2.1, it can be seen that the model consistently under estimates the actual value of Covid cases per 100k people. This could occur for a variety of different reasons, but typically, this could be because the model is specified incorrectly potentially from the log transformations or linearity being an overall bad fit; there are also likely very important factors that are omitted including but not limited to testing rate, policy strictness, mobility etc. . . ; there are also potentially other interaction terms that are significant but were not tested for.

For MLR the four main assumptions are to ensure the model is accurate and the predictor coefficients are reliable are:

- Linearity
- Normality
- Homoscedasticity
- Independence

The plots below will be used to assess the validity of some of these assumptions. Starting with the

```
plot(covid_lm4, which = 1)
```

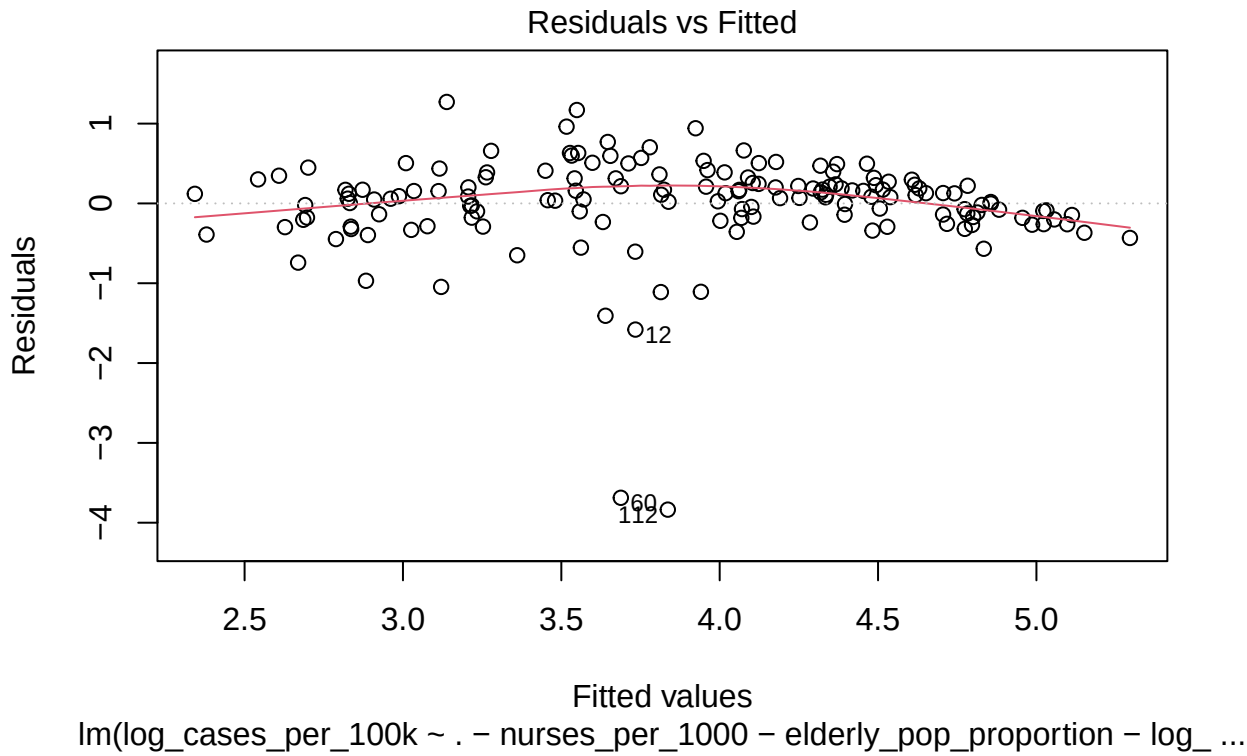


Figure 2.2: Residuals (actual values - model's observed values) vs fitted values plot to check for linearity

In Figure 2.2 the majority of points appears to be randomly scatted above and below the zero-line with no clear pattern, however there seems to be a slight grouping trend with the values to the right side of the plot, there are also a few outliers in the middle of the plot although this can be seen as negligible. Overall, the assumption of linearity appears. slightly violated.

```
plot(covid_lm4, which = 3)
```

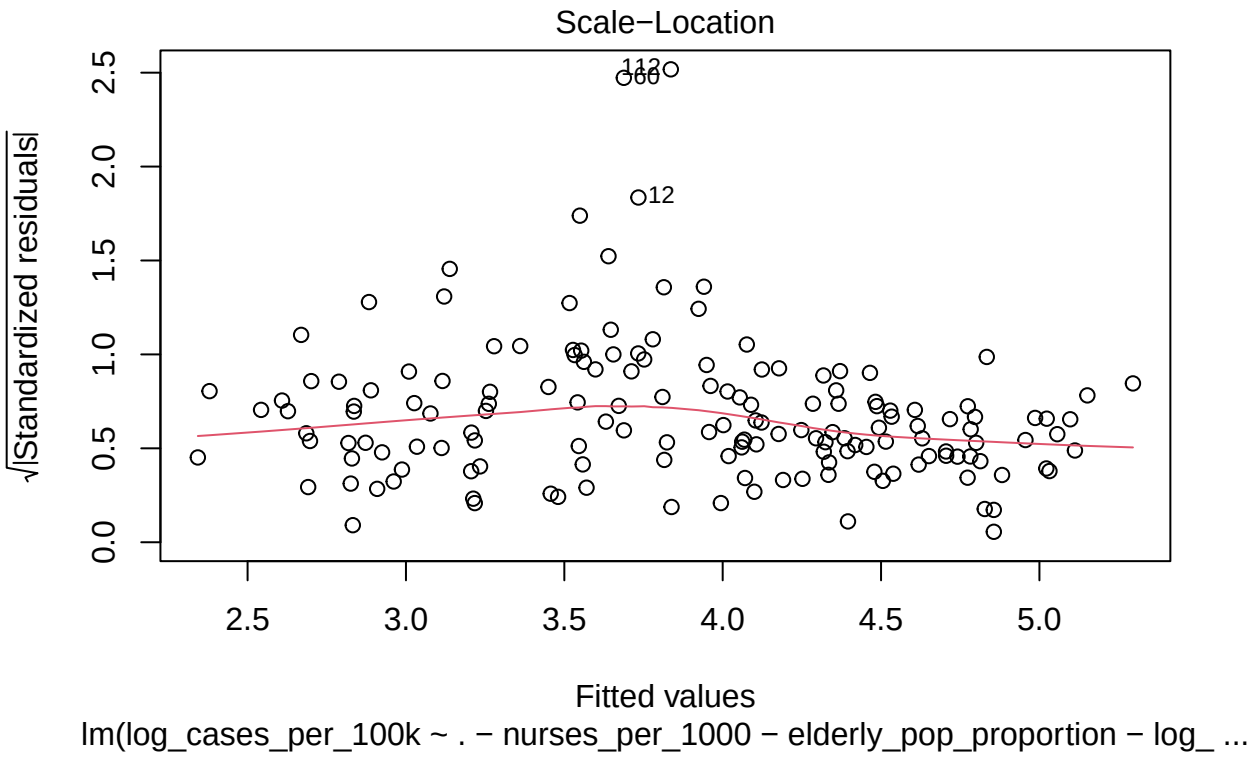


Figure 2.3: Scale location plot to test for homoscedasticity

Shown in the scale-location plot in Figure 2.3, the assumption of homoscedasticity seems to be valid, the red line trend line is relatively flat and there are a consistent random scatter of points above and below it with no clear pattern.

```
plot(covid_lm4, which = 2)
```

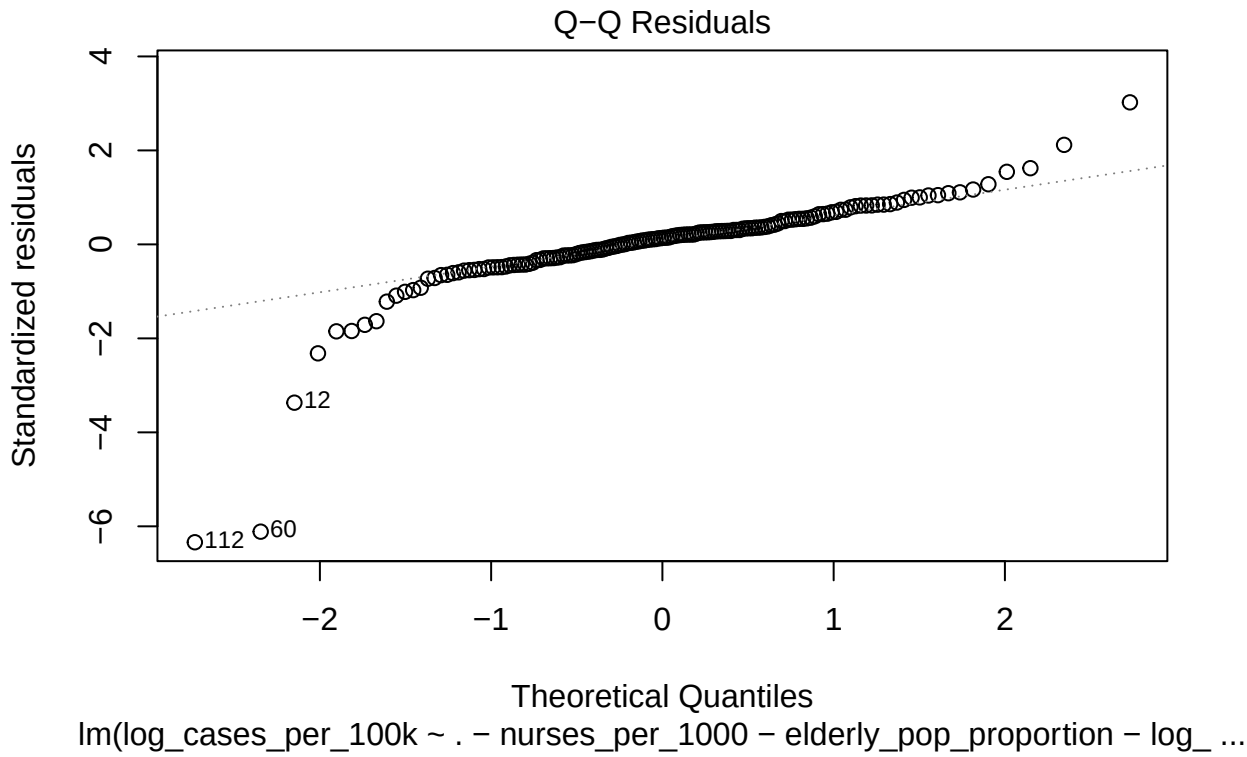


Figure 2.4: QQ plot to test for normality

Figure 2.4 shows that the majority of points lie on the 45 degree line with some deviations at the tails, a few points in particular have very large deviations. This shows that the majority of the residuals are approximately normally distributed, however, there does seem to be significant outliers. Due to this, the assumption is only marginally valid.

Conclusions:

- Linearity: Partially invalid
- Homoscedasticity: Valid
- Normality: Mainly valid with some outliers
- Independence: To truly confirm this assumption, information on how the data is collected is required. The aggregated_data data frame treats each country as its own self contained observation where independence between the residuals of each observation is technically plausible. However, the assumption is likely invalid since realistically neighboring countries or countries with similar policies will likely be dependent observations.

Overall, the model is reasonably reliable, however, the model could be potentially improved with different types of variable transformations that would better validate the linearity assumption. In terms of precision, the model has moderate strength with an R^2 of 0.5923, speaking generally for social, health, and economic data since there are so many different underlying factors that are not included in the model an R^2 of 0.5-0.6 seems reasonable. Overall, the model can give meaningful insights, however there is still room to improve on. The RMSE calculated previously will help to compare this model with another model in this investigation

Regression Tree

This part of the investigation will apply a regression tree model to try and predict covid_cases_per_100k using the same predictor variables. To do this the rpart and vip packages will be used to run and analyse the regression tree model.

The code below splits the aggregated data set into a prediction data set for the regression tree model. The process of splitting, defining the recipe, using `juice()` and `bake()` functions to specify the pre-processed are the exact same as the linear regression model, just without log transformations. Unlike multiple linear regression, regression tree does not assume linearity, therefore the log transformed variables may help stabilize variance but overall is not necessary.

```
pacman::p_load(rpart, rpart.plot, vip)

set.seed(111)
rt_predictiondata <- aggregated_data %>%
  select(cases_per_100k, gdp_per_capita_usd,
         health_expenditure_usd,
         nurses_per_1000, population_urban,
         elderly_pop_proportion, population_density,
         life_expectancy, smoking_prevalence,
         region)

rt_split <- initial_split(rt_predictiondata)
rt_split
```

```
## <Training/Testing/Total>
## <158/53/211>
```

```
rt_train <- training(rt_split)
rt_test <- testing(rt_split)


rt_recipe <- recipe(cases_per_100k ~., data = rt_train) %>%
  step_dummy(all_nominal(), - all_outcomes())

rt_recipe_prepped <- rt_recipe %>%
  prep()

rt_train_preproc <- juice(rt_recipe_prepped)
rt_test_preproc <- bake(rt_recipe_prepped, rt_test)
```

The code below specifies the model and plots the regression tree. The extra parameters complexity parameter (cp) controls how much each addtional split in the tree must improve the model's fit by in terms of residual sum of squares; minsplit is the minimum number of observations required in a node before it can split to avoid very small and unrelaible split in the regression tree.

```
tree_model <- rpart(
  cases_per_100k ~ .,
  data = rt_train_preproc,
  method = "anova",   # Using 'anova' for continuous outcomes
  control = rpart.control(cp = 0.01, minsplit = 15)
)
```

```
rpart.plot(tree_model, type = 3, extra = 101, fallen.leaves = TRUE)
```

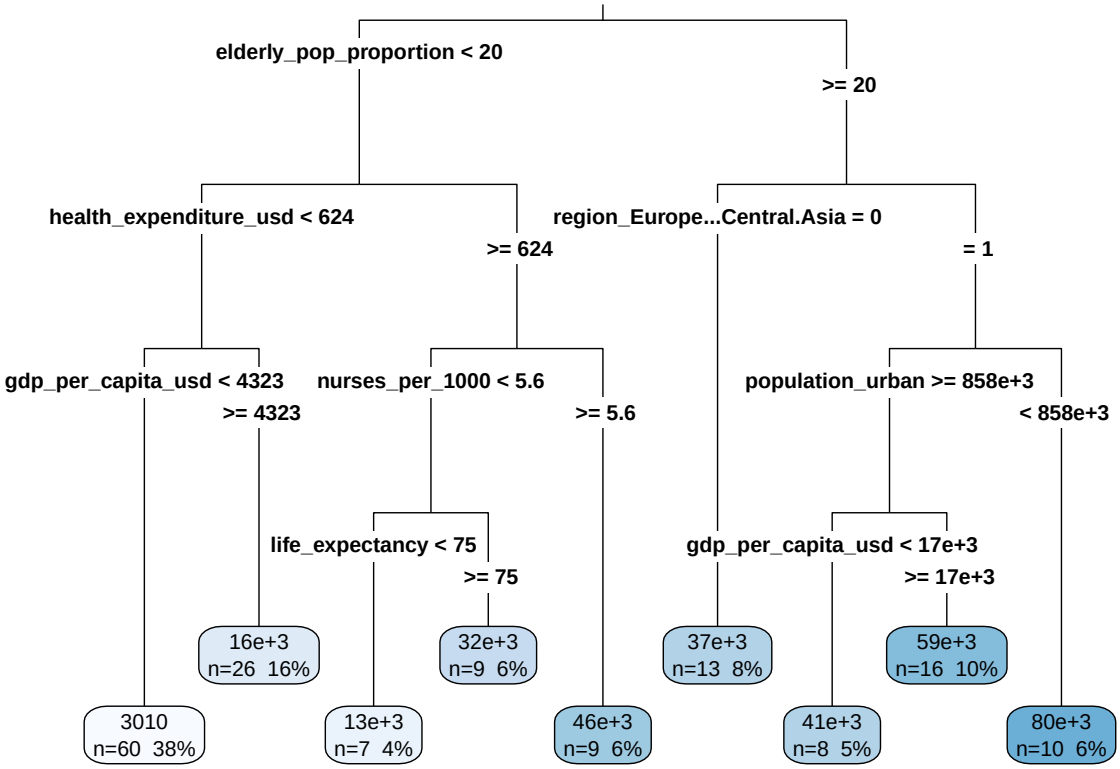


Figure 2.5: The regression tree plot made using the training data specified previously from the covidDf aggregated_data dataset. The 14 predictor variables that were listed for the MLR model is also included here (8 numerical variables and 6 dummy variables for region).

The code below plots the variable importance plot (VIP).

```
tree_model %>%
  vip(num_features = 14) + theme_minimal() +
  labs(title = "VIP plot - Regression Tree")
```

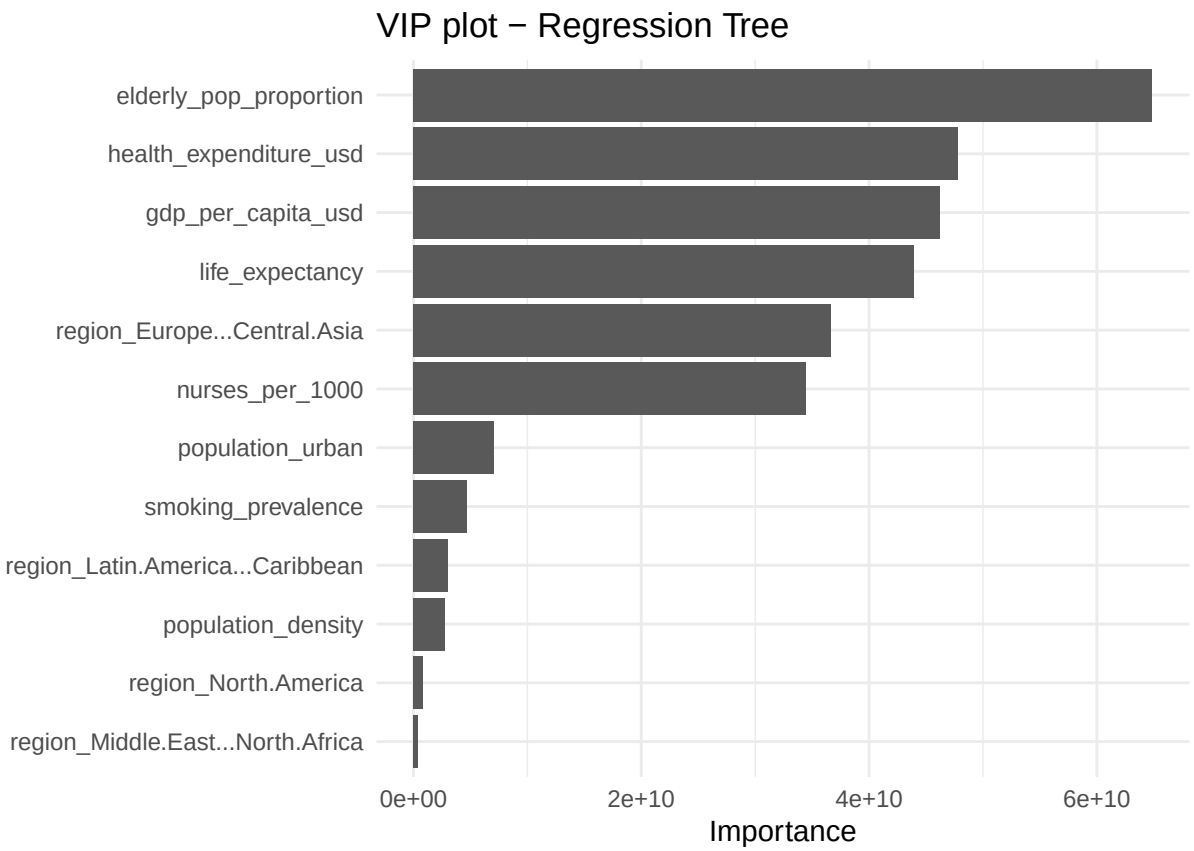


Figure 2.6: VIP plot for the regression tree model. The plot shows the which predictors are the most important determined by how much they reduce error and improve prediction accuracy. Shown here on this plot that a countries' elderly population proportion is the most important variable in determining deaths per 100k people.

The code below uses the regression tree model to predict the values of the testing data set. The yardstick R package and metrics() function is used to automatically calculate the RMSE (14126) and r^2 (0.5867) values shown in the output table below.

```
case_per_100k_pred <- rt_test_preproc %>%
  mutate(.pred = predict(tree_model, newdata = rt_test_preproc))

library(yardstick)
case_per_100k_pred %>%
  metrics(truth = cases_per_100k, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard   14126.
## 2 rsq     standard       0.587
## 3 mae     standard   10237.
```

Plotting the predicted vs actual values plot for the regression tree model

```
rt_pvs <-  case_per_100k_pred %>% ggplot(aes(x = `cases_per_100k`, y = `.pred`)) +
      geom_point() +
      geom_smooth(method = "lm") +
      geom_abline(intercept=0, slope = 1, linetype="dashed", color = "red") +
      labs(title = "Predicted vs Actual Values - Regression Tree",
           x = "Actual Values",
           y = "Predicted Values") +
      theme_minimal()

rt_pvs
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

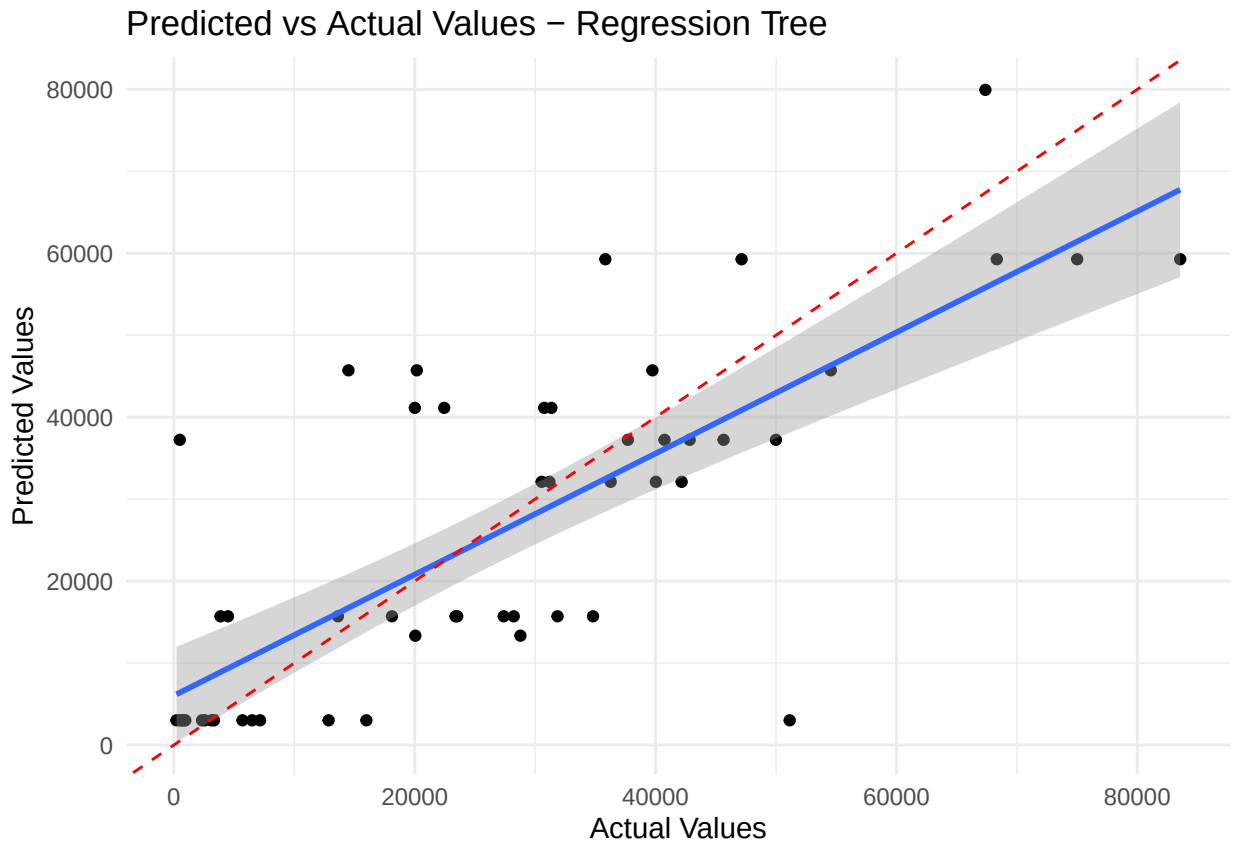


Figure 2.7: Predicted vs actual values scatter plot for the regression tree model. Shown in the plot there is a decent amount of variance in the points. Most points on the are close the the red dotted line which shows that the prediction was accurate, there are also some outlier predictions that heavily underestimated or overestimated the ratio of covid deaths per 100k people.

Random Forests

The performance of the regression trees vs linear regression will be discussed in more detail later, however, the simple regression trees model outperformed MLR in some regards while did a little worse in others. Therefore, random forests with hyper parameter tuning will be the final predictive method used in this investigation as a way to explore a more refined regression tree model's performance. Similarly, this method will be used to assess how well the 14 variables (including dummy variables for region) listed previously

for MLR and regression trees perform at predicting covid cases per 100k people for a given country. Since a random forest is made up of multiple regression trees, the same training and testing sets made for the regression tree model `rt_train` and `rt_test` will be used for this predictive model as well.

However, for this model instead of manually assigning reasonable parameters similar to the regression tree model, some tuning will be done to calculate the optimal mtry and min_n parameter values for the model.

- mtry: When calculating each tree, at a split mtry is the number of variables that will be randomly sampled
- min_n: A split in a tree is only allowed if the node where the split is happening have a least min_n observations

Other things to note is that in the model specification code below, the other parameters e.g., trees are set to 500 so the model will train on 500 different trees; the engine is set the "ranger" which comes from the {ranger} package, it stands for Rapid Random Forest Generator and is designed for faster implementation of the random forest model for more complex datasets; finally the importance parameter is set to "permutation" which means the model will take into account the importance of each variable in the random forest.

```
pacman::p_load(tune, ranger)
set.seed(111)



rf_spec <- rand_forest(
  mode = "regression",
  trees = 500,
  mtry = tune(),
  min_n = tune()
) %>%
  set_engine("ranger", importance = "permutation")
```

Using the vfold_cv() function, the training data is split into 5 folds. Additionally a parameters grid that contains all possible combinations based on the given parameters of mtry and min_n are created using the grid_regular function(). The finalize() function adjusts mtry to the maximum possible value based on the data excluding cases_per_100k since it is the target variable. The default min_n() is used for the min_n parameter and levels is set to 5 which means 5 equally spaced values are created for each parameter.

```
set.seed(111)
covid_cv <- vfold_cv(rt_train, v = 5)

params_grid <- grid_regular( finalize( mtry(), rt_train %>% select( -cases_per_100k ) ),
                             min_n(),
                             levels = 5)

rf_tuned <- tune_grid( object = rf_spec,
                       preprocessor = rt_recipe,
                       resamples = covid_cv,
                       grid = params_grid )
```

Here the select_best() function returns the combination of mtry and min_n that provides the lowest RMSE. The finalize_model() function updates the random forest model's specification (rf_spec) with the optimal combination of mtry and min_n found previously. Then the model is fitted with the training data for further assessment of the model's performance.

```
best_rmse <- select_best( rf_tuned, metric="rmse" )
final_rf <- finalize_model( rf_spec, best_rmse )

covid_rf <- final_rf %>%
  fit( cases_per_100k ~ . , data = rt_train )
```

```
covid_rf %>%
  vip(num_features = 14) +
  labs(title = "VIP plot - Random Forest") +
  theme_minimal()
```

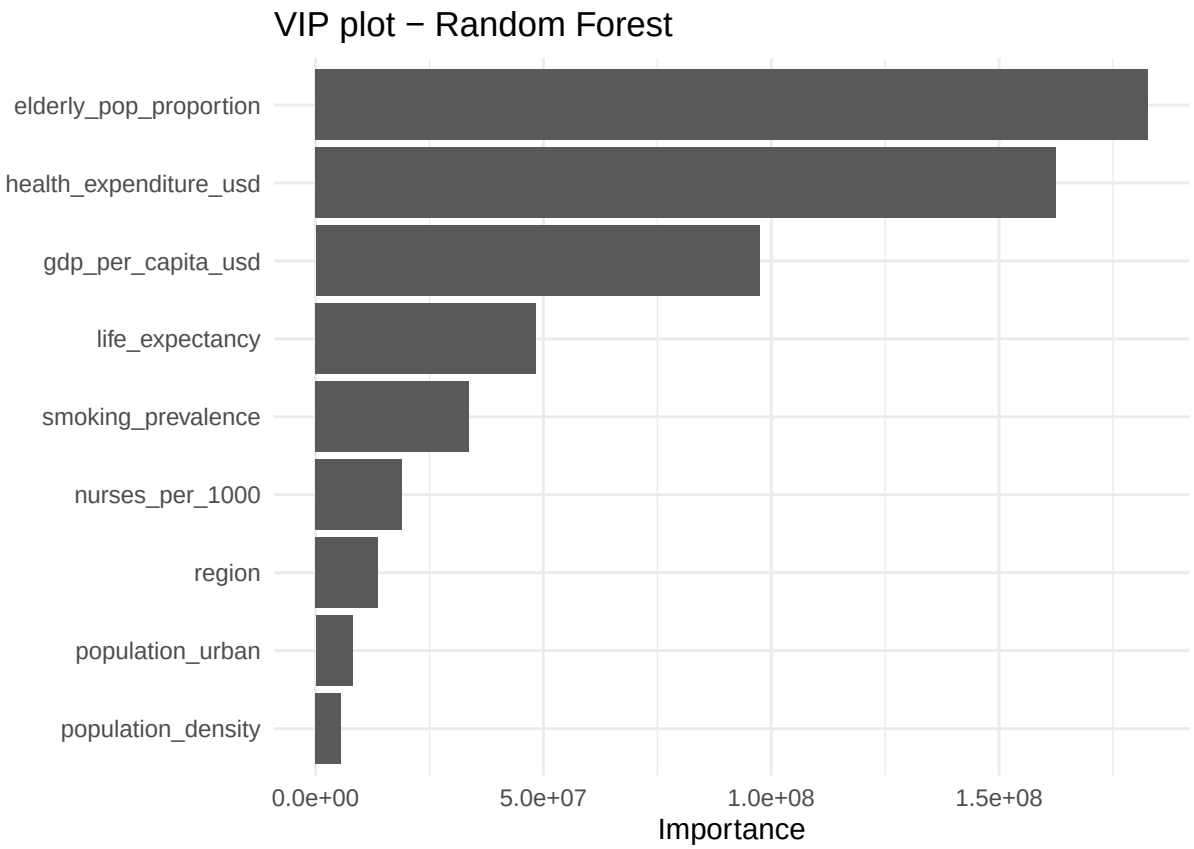


Figure 2.8: VIP plot for the random forest model. The plot shows the which predictors are the most important determined by how much they reduce error and improve prediction accuracy.

Figure 2.8's VIP plot is relatively similar to the regression tree model's VIP plot (Figure 2.6). The majority of the variables rank the same in terms of importance. A notable difference for the random forest VIP plot is that health expenditure is a lot closer to elderly population proportion in terms of importance, and the decrease in importance for each variable resembles a smooth exponential decrease as opposed to sharper decreases in the regression tree model's variables. This does seem to reflect the general nature of the two models as for regression tree, only a few variables are chosen for top splits and importance drops off sharply, random forest are built on bootstrapped samples so variables with less importance can still show up and potentially contribute more.

Predicting cases per 100k people using the random forest model and calculating RMSE (12661) and r^2 (0.6413).

```
case_per_100k_pred_rf <- rt_test %>%
  mutate(.pred = predict(covid_rf, new_data = rt_test) %>% pull(.pred))


case_per_100k_pred_rf %>%
  metrics(truth = cases_per_100k, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard   12661.
## 2 rsq     standard       0.641
## 3 mae     standard    8366.
```

Plotting the predicted vs actual values plot for the random forest model using ggplot.

```
rf_pvs <-  case_per_100k_pred_rf %>% ggplot(aes(x = `cases_per_100k`, y = `.pred`)) +
      geom_point() +
      geom_smooth(method = "lm") +
      geom_abline(intercept=0, slope = 1, linetype="dashed", color = "red") +
      labs(title = "Predicted vs Actual Values - Random Forest",
           x = "Actual Values",
           y = "Predicted Values") +
      theme_minimal()

rf_pvs
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

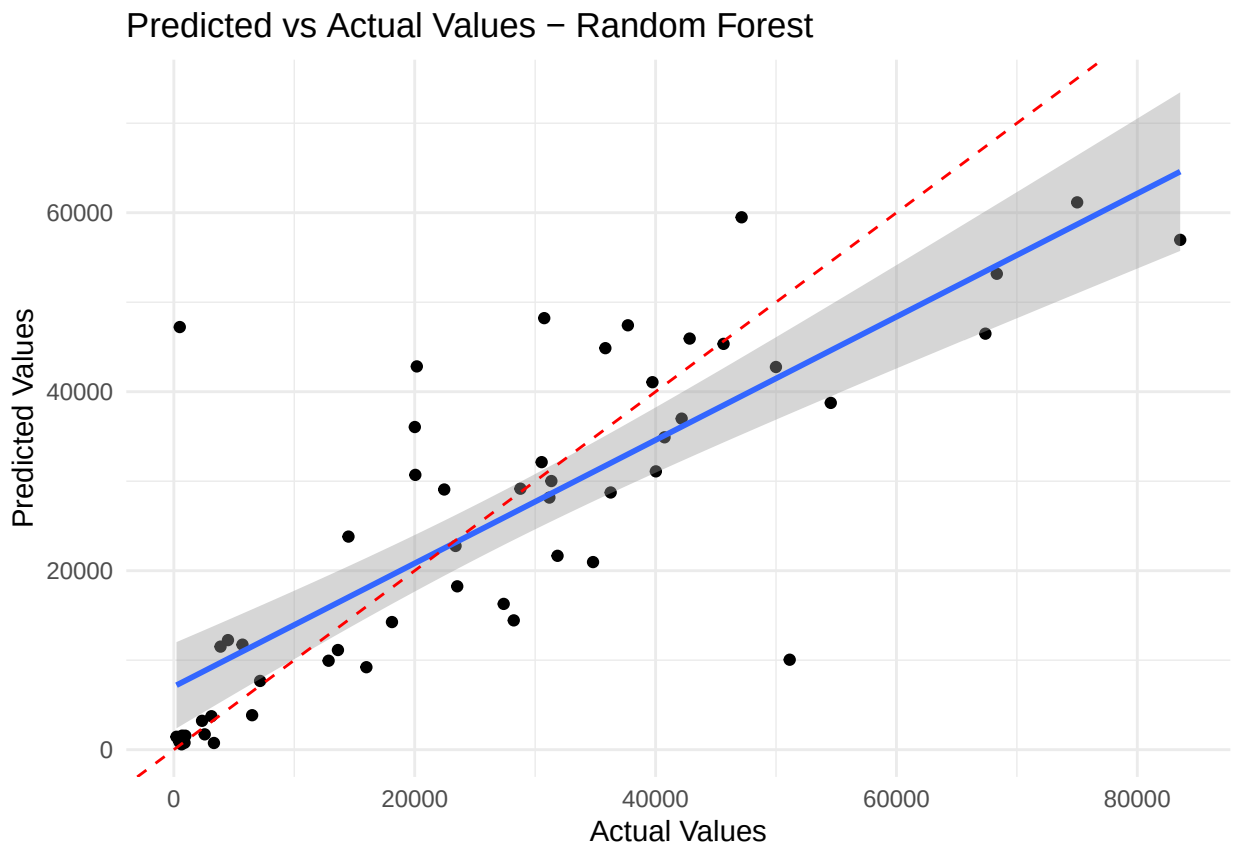


Figure 2.9: Predicted vs actual values scatter plot for the random forest model. The plot here is quite similar to the regression tree model's predicted vs actual values plot. However, here the points seem more tightly packed around the red dotted line which showing more accurate predictions and slightly less significant outliers.

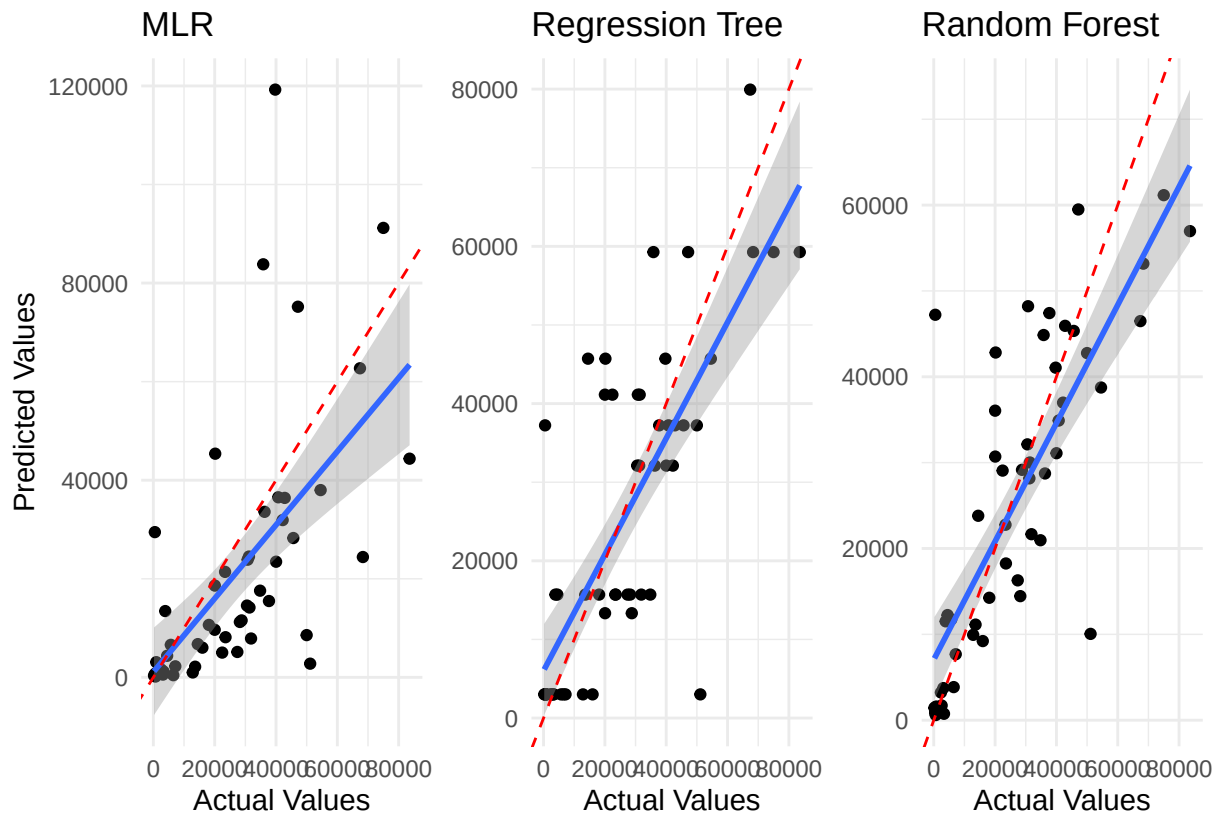
Comparison of the Models

To give a final answer as to *which model should be chosen?* This investigation will summarise and draw some final comparisons from the predicted vs actual values plot, R^2 values and RMSE as shown below.

```
#Reformatting the plot titles to make the combined plot clearer
rt_pvs <- rt_pvs + labs(
  y = NULL,
  title = "Regression Tree"
)
rf_pvs <- rf_pvs + labs(
  y = NULL,
  title = "Random Forest"
)
mlr_pvs <- predictedvsac_plot + labs(
  title = "MLR"
)

#Combining the three plots previously made using patchwork library
mlr_pvs + rt_pvs + rf_pvs
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```



Shown in the comparison plots above, all of the models seems to generally underestimate the value of Covid cases per 100k, the random forest model appears to do it the least. It is clear that the points tend to scatter closer to the perfect prediction line for both the regression tree and random forest model compared to the MLR model. However, random forest seems to have points and the blue trend line align most tightly clustered with the perfect predictions line. Additionally, regression tree model shows consistent horizontal groupings of points, so the predictions are not smooth unlike MLR or random forest, which shows that regression trees can capture general patterns well but not be able to represent continuous variability in the data. For these reasons, the random forest model is the best here.

The code below produces a table that summarises all of the RMSE and R^2 values of the different models calculated previously in the investigation.

```
model_results <- data.frame(
  Model = c("Multiple Linear Regression", "Regression Tree", "Random Forest"),
  RMSE = c(21328.82, 14126.46, 12660.69),
  R_Squared = c(0.5923, 0.5867, 0.6413)
)

# Display it nicely as a table
kable(model_results,
      caption = "Model Performance Comparison",
      digits = 3,
      col.names = c("Model", "RMSE", "R²"))
```

Table 2: Model Performance Comparison

Model	RMSE	R^2
Multiple Linear Regression	21328.82	0.592
Regression Tree	14126.46	0.587
Random Forest	12660.69	0.641

The MLR model has a higher R^2 but a lower RMSE compared to regression trees. This meaning that the MLR model is better than regression trees at making predictions close observed values but less consistent and makes larger errors per prediction. However, this is also to say that out of the three models the random forest model did the best in both regards.

In conclusion, the best performing model at predicting Covid cases per 100k people for a given country from 2020-2022 out of the three is the random forest model. This is shown through the predicted vs actual values plot; the highest R^2 which means the model is the best at explaining the variation in the response variable; the lowest RMSE value which measures the average prediction error between predicted and actual values of Covid cases per 100k people.